



PROGRAMACIÓN COMPETITIVA

Notebook

Python

Entrada/Salida • Estructuras de Datos • Grafos
Búsqueda y Optimización • Matemáticas y Simulación
Construcción y Combinatoria • Cadenas

Notebook de referencia para
ICPC • Codeforces • UVA
CSES • Gym

Compilado y mantenido por
David Santiago García Preciado

Ingeniería de Sistemas
Universidad El Bosque

Versión 1.0
Última actualización
20 de septiembre de 2026

Índice

1	Entrada/Salida Rápida en Python	2			
1.1	Lectura línea a línea (caso general) .	2			
1.2	Lectura total + tokenización (máxima velocidad)	2			
1.3	Casos con bandera (condicional) . . .	2			
1.4	Casos hasta EOF (sin contador T) . .	2			
1.5	Salida rápida	2			
2	Herramientas y Funciones Esenciales de Python	3			
2.1	Iteradores y funciones integradas . . .	3			
2.2	Expresiones Regulares (módulo re) .	4			
2.3	Diccionarios: dict, defaultdict y Counter	5			
2.4	Manipulación de cadenas	7			
2.5	Módulos matemáticos	8			
2.6	Librerías estándar útiles para ICPC .	10			
3	Estructuras de Datos	13			
3.1	Fenwick Tree / BIT + Consultas Offline (distintos en un rango)	13			
3.2	Prefix Sums + Mapa de Primera Aparición (subarreglo balanceado más largo)	13			
3.3	Fenwick Tree con XOR + Sprague-Grundy (Nim por rangos)	14			
4	Algoritmos de Grafos	16			
4.1	Emparejamiento Bipartito Máximo, Kuhn (parejas de baile)	16			
4.2	Kuhn sobre Segmentos de Fila/Columna (torres con peones) . .	16			
4.3	Teorema de König: Cobertura Mínima de Vértices (quitar caballos) . . .	17			
4.4	Componentes Fuertemente Conexas, Kosaraju (reinos de planetas)	18			
4.5	Circuito Euleriano, Hierholzer (ruta postal)	19			
4.6	Puntos de Articulación, Tarjan (islas críticas)	19			
4.7	Bron-Kerbosch: Clique Máximo (grupo compatible)	20			
4.8	Orden Topológico, Kahn + Propagación de conteos (Digi Comp II) . . .	21			
4.9	Union-Find / DSU con Path Halving (componentes conexas)	22			
5	Búsqueda y Optimización	24			
5.1	Búsqueda Binaria sobre la Respuesta + Two Pointers (k-ésima suma de pares)	24			
5.2	Enumeración por Bitmask (bits perdidos de un mensaje)	24			
5.3	Búsqueda Binaria sobre la Respuesta: Maximizar el Mínimo (muralla con refuerzo)	25			
5.4	Huffman: Fusión Greedy con Heap (teñir telas)	26			
5.5	DP de Particionamiento sobre Residuos (compra con redondeo)	26			
6	Matemáticas y Simulación	28			
6.1	Reducción Binaria tipo Josefo (fila con detector alternado)	28			
6.2	Fuerza Bruta / Simulación Directa (paradoja del cumpleaños)	28			
6.3	Simulación Lineal con Máximo de Prefijo (paredes que se acercan) . . .	28			
6.4	Simulación de Eventos con Colas (escalera doble)	29			
6.5	Simulación XOR con Detección de Ciclos (interruptores y bombillas) . .	30			
7	Construcción y Combinatoria	31			
7.1	Constructivo por Invariante de Paridad (permutación con signos)	31			
7.2	Constructivo por Construcción Explícita (suma divisible por cada elemento)	31			
7.3	Conteo por Simetría de Binomios (subsecuencias con suma alternada 0) .	32			
7.4	Greedy por Alternancia con Límite de Racha (dónde dormir cada noche) .	32			
7.5	Stars and Bars con Factoriales Precalculados (días de espera entre oasis) .	33			
8	Algoritmos de Cadenas (Strings)	34			
8.1	Balance de Paréntesis por Bloques de Nivel Superior (parejas incompatibles) .	34			
8.2	Rotación Mínima, Booth (huellas duplicadas por rotación)	34			
8.3	Pila con Restricción de Capacidad (matrioshkas)	35			
8.4	Normalización + Comparación con el Reverso (palíndromos)	35			
8.5	Reducción con Pila (ciclos en un bang path)	36			
8.6	KMP sobre Codificación en Preorden (subárboles con la misma forma) . . .	37			
8.7	Suffix Array + LCP de Kasai (subcadena repetida más larga)	38			
8.8	Firma Canónica + Hash Map (palabras con letras revueltas)	39			
8.9	Regex: findall y lookahead (conteo de patrones)	40			

Este Notebook tiene una estructura simple. Cada entrada lleva el nombre de la técnica y, entre paréntesis, el problema con el que se ilustra; y siempre tiene las mismas partes: Preguntas guía para reconocer cuándo aplica, Utilidad, Complejidad, Fundamento teórico, Ejercicio aplicado, I/O (cómo se leyó la entrada y qué patrón de la sección 1 se usó) y Código solución verificado en juez.

1 Entrada/Salida Rápida en Python

En Python el I/O es el cuello de botella más común en ICPC. Estas son las 4 situaciones típicas y el patrón recomendado para cada una.

1.1 Lectura línea a línea (caso general)

Cuando cada línea tiene una cantidad de datos conocida y el número de líneas no es extremo (hasta 10^5). Es el patrón por defecto.

```
import sys
input = sys.stdin.buffer.readline
n = int(input())
arr = list(map(int, input().split()))
```

1.2 Lectura total + tokenización (máxima velocidad)

Cuando hay muchísimos números sueltos (10^5 – 10^6 +) sin importar en cuántas líneas están repartidos. Es la forma más rápida en Python, ya que es una sola llamada al sistema.

```
import sys
data = sys.stdin.buffer.read().split()
idx = 0 # el idx es lo que recorre el arreglo de tokens
n = int(data[idx]); idx += 1
arr = list(map(int, data[idx:idx + n])); idx += n
```

1.3 Casos con bandera (condicional)

El input termina cuando aparece un valor en concreto. No se sabe de antemano cuántos casos hay.

```
import sys
input = sys.stdin.buffer.readline
while True:
    a, b = map(int, input().split())
    if a == 0 and b == 0: # en este caso el valor
                           concreto es "0 0"
        break
    # aquí se procesan los casos
```

1.4 Casos hasta EOF (sin contador T)

No hay número de casos declarado, por lo que se procesa hasta que no haya mas inputs.

```
import sys
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    # se procesa el caso
```

Una alternativa con los casos infinitos es usar el metodo de tokenización.

```
import sys
data = sys.stdin.buffer.read().split()
idx = 0
while idx < len(data):
    # se procesa el caso
```

1.5 Salida rápida

Nunca usar print() y mejor acumular resultados en una lista y escribir todo de una sola vez.

```
ans = []
# ans.append(result) dentro del loop
sys.stdout.write("\n".join(map(str, ans)) + "\n")
```

2 Herramientas y Funciones Esenciales de Python

2.1 Iteradores y funciones integradas

Estas funciones permiten recorrer estructuras de datos, transformar secuencias y escribir código más corto y legible sin sacrificar rendimiento.

2.1.1 enumerate()

Recorrer un iterable obteniendo simultáneamente el índice y el elemento, evitando mantener un contador manual.

```
arr = [5, 8, 2]
for i, x in enumerate(arr):
    print(i, x)
for i, x in enumerate(arr, 1):      # índices comienzan en 1
    print(i, x)
```

Es especialmente útil cuando el problema utiliza índices 1-indexados.

2.1.2 zip()

Recorrer múltiples iterables en paralelo. El recorrido termina cuando el iterable más corto finaliza.

```
a = [1, 2, 3]
b = [4, 5, 6]
for x, y in zip(a, b):
    print(x, y)

pares = list(zip(a, b))
# [(1,4), (2,5), (3,6)]
```

También puede utilizarse para desempaquetar listas de tuplas.

```
pares = [(1, 4), (2, 5), (3, 6)]
a, b = zip(*pares)
print(a); print(b)
```

2.1.3 iter() y next()

iter() crea un iterador sobre cualquier iterable. next() consume el siguiente elemento del iterador. Se suelen usar cuando se procesan tokens manualmente.

```
data = ["5", "10", "20", "30"]
it = iter(data)
n = int(next(it))
arr = [int(next(it)) for _ in range(n - 1)]
```

También puede especificarse un valor por defecto.

```
x = next(it, None)
```

2.1.4 any() y all()

Evaluar rápidamente condiciones sobre un iterable.

any() devuelve True si al menos un elemento cumple la condición. all() devuelve True únicamente si todos los elementos la cumplen. Ambas realizan *short-circuit*, por lo que dejan de recorrer el iterable tan pronto conocen la respuesta.

```
if any(x % 2 == 0 for x in arr):
    print("Existe un numero par")
if all(x > 0 for x in arr):
    print("Todos son positivos")
```

2.1.5 map()

Aplica una función a todos los elementos de un iterable. Es muy utilizado durante la lectura de entrada.

```
nums = list(map(int, input().split()))
cuadrados = list(map(lambda x: x * x, nums))
```

También puede utilizarse con funciones propias.

```
def f(x):
    return x + 1
nuevo = list(map(f, nums))
```

2.1.6 sorted()

Devuelve una nueva lista ordenada sin modificar la original.

Puede ordenarse mediante una función key.

```
nums = sorted(nums)
nums = sorted(nums, reverse=True)
personas = sorted(personas, key=lambda x: x.edad)
pares = sorted(freq.items(), key=lambda kv: -kv[1])
```

Complejidad: $\mathcal{O}(n \log n)$

Python utiliza Timsort, un algoritmo estable muy eficiente para datos parcialmente ordenados.

2.1.7 reversed()

Permite recorrer una secuencia en orden inverso sin crear una copia.

```
for x in reversed(arr):
    print(x)
```

Es preferible a lo siguiente, que crea una copia de la lista:

```
arr[::-1]
```

2.1.8 encode() y decode()

Aunque rara vez aparecen en ICPC, pueden resultar útiles cuando un problema involucra archivos, comunicación o codificaciones específicas.

encode() convierte un string en bytes. decode() realiza la operación inversa.

```
s = "ICPC"
b = s.encode("utf-8")
print(b)
# b'ICPC'
texto = b.decode("utf-8")
print(texto)
# ICPC
```

Las codificaciones más comunes:

- utf-8
- ascii

- latin-1

En programación competitiva normalmente toda la entrada ya llega como texto UTF-8, por lo que estas funciones aparecen con poca frecuencia.

Resumen

Función	Uso principal
<code>enumerate()</code>	Recorrer con índice
<code>zip()</code>	Recorrer varios iterables
<code>iter()</code>	Crear un iterador
<code>next()</code>	Consumir siguiente elemento
<code>any()</code>	Existe algún elemento que cumpla
<code>all()</code>	Todos cumplen
<code>map()</code>	Aplicar función a todos los elementos
<code>sorted()</code>	Ordenar sin modificar el original
<code>reversed()</code>	Recorrer en orden inverso
<code>encode()</code>	String → bytes
<code>decode()</code>	Bytes → string

2.2 Expresiones Regulares (módulo re)

El módulo `re` permite buscar, validar, reemplazar y extraer información de cadenas mediante patrones.

En programación competitiva suele utilizarse cuando la entrada posee un formato irregular, contiene separadores complejos o es necesario validar cadenas.

```
import re
```

2.2.1 Funciones principales

Función	Descripción
<code>re.match()</code>	Busca únicamente al inicio del string.
<code>re.search()</code>	Busca la 1ra coincidencia en cualquier posición.
<code>re.findall()</code>	Devuelve todas las coincidencias en una lista.
<code>re.finditer()</code>	Devuelve un iter con todos los matches.
<code>re.fullmatch()</code>	Verifica que toda la cadena cumpla patrón.
<code>re.sub()</code>	Reemplaza coincidencias.
<code>re.split()</code>	Divide una cadena utilizando un patrón.
<code>re.compile()</code>	Compila una regex para reutilizarla.

2.2.2 Símbolos más utilizados

Patrón	Significado
<code>.</code>	Cualquier carácter excepto salto de línea
<code>\d</code>	Dígito
<code>\D</code>	No dígito
<code>\w</code>	Letra, número o <code>_</code>
<code>\W</code>	No alfanumérico
<code>\s</code>	Espacio en blanco
<code>\S</code>	No espacio
<code>+</code>	Una o más repeticiones
<code>*</code>	Cero o más repeticiones
<code>?</code>	Cero o una repetición
<code>{n}</code>	Exactamente n repeticiones
<code>{n,m}</code>	Entre n y m repeticiones
<code>[]</code>	Conjunto de caracteres
<code>[^]</code>	Negación del conjunto
<code>()</code>	Grupo de captura
<code>(?:)</code>	Grupo sin captura
<code> </code>	OR lógico
<code>^</code>	Inicio del string
<code>\$</code>	Final del string

2.2.3 Ejemplos básicos

```
import re
nums = re.findall(r'~?\d+', s)
nums = list(map(int, nums))
tokens = re.split(r'\s+', s.strip())
m = re.match(r'^(\d+)-(\d+)$', s)
if m:
    a = int(m.group(1))
    b = int(m.group(2))
limpio = re.sub(r'[^a-zA-Z0-9]', '', s)
```

2.2.4 Patrones útiles para ICPC

Patrón	Detecta
<code>\d+</code>	Enteros positivos
<code>-?\d+</code>	Enteros con signo
<code>\d+\.\d+</code>	Números decimales
<code>[A-Za-z]+</code>	Palabras
<code>[A-Za-z0-9]+</code>	Alfanuméricos
<code>[aeiou]</code>	Vocales
<code>[^aeiou]</code>	Todo excepto vocales
<code>\s+</code>	Uno o más espacios
<code>\w+</code>	Identificadores
<code>.+</code>	Cualquier texto

2.2.5 Combinaciones frecuentes

Extraer todos los enteros.

```
re.findall(r'~?\d+', s)
```

Extraer únicamente números positivos.

```
re.findall(r'\d+', s)
```

Extraer palabras.

```
re.findall(r'[A-Za-z]+', s)
```

Extraer palabras y números.

```
re.findall(r'\w+', s)
```

Separar por múltiples espacios.

```
re.split(r'\s+', s.strip())
```

Eliminar caracteres especiales.

```
re.sub(r'^A-Za-z0-9', '', s)
```

Eliminar espacios repetidos.

```
re.sub(r'\s+', ' ', s)
```

Validar un correo electrónico sencillo.

```
re.fullmatch(r'\w+@\w+\.\w+', s)
```

Validar una fecha DD/MM/AAAA.

```
re.fullmatch(r'\d{2}/\d{2}/\d{4}', s)
```

Validar un entero.

```
re.fullmatch(r'?-\d+', s)
```

Buscar únicamente al inicio.

```
re.match(r'ICPC', s)
```

Buscar en cualquier posición.

```
re.search(r'ICPC', s)
```

2.2.6 Compilar una expresión regular

Cuando un mismo patrón será utilizado muchas veces, es recomendable compilarlo una única vez. Esto evita recompilar la expresión regular en cada llamada.

```
pat = re.compile(r'?-\d+')
for linea in texto:
    nums = pat.findall(linea)
```

2.2.7 Complejidad

La mayoría de operaciones tienen complejidad aproximadamente $\mathcal{O}(n)$ respecto al tamaño del texto, aunque algunos patrones muy complejos pueden provocar retroceso (*backtracking*) y aumentar considerablemente el tiempo de ejecución.

En problemas de ICPC normalmente se utilizan patrones simples, por lo que el rendimiento suele ser lineal.

Resumen

Función	Uso principal
match	Buscar al inicio
search	Buscar primera coincidencia
findall	Obtener todas las coincidencias
finditer	Iterar sobre coincidencias
fullmatch	Validar una cadena completa
split	Separar mediante un patrón
sub	Reemplazar texto
compile	Reutilizar un patrón

2.3 Diccionarios: dict, defaultdict y Counter

Los diccionarios son una de las estructuras de datos más utilizadas en programación competitiva. Permiten almacenar pares **clave** : **valor** con acceso promedio en tiempo constante.

En Python, la implementación utiliza una tabla hash, por lo que la mayoría de operaciones tienen complejidad promedio $\mathcal{O}(1)$ aunque en el peor caso pueden degradarse a $\mathcal{O}(n)$

si existen muchas colisiones.

2.3.1 Crear un diccionario

```
freq = {}
persona = {"nombre": "David", "edad": 20}
```

2.3.2 Acceso y recuperación

Acceder directamente mediante corchetes genera un **KeyError** si la clave no existe.

```
x = freq["a"]
```

Por ello normalmente es preferible utilizar `get()`.

```
freq.get("a")
freq.get("a", 0)
freq.get("a", [])
```

Las funciones principales de recuperación son:

Función	Descripción
get()	Obtiene un valor sin lanzar excepción.
keys()	Devuelve todas las claves.
values()	Devuelve todos los valores.
items()	Devuelve pares (clave, valor).

Ejemplos:

```
for k in freq.keys():
    ...
for v in freq.values():
    ...
for k, v in freq.items():
    ...
```

2.3.3 Modificar un diccionario

Insertar o modificar una clave.

```
freq["a"] = 5
```

Actualizar múltiples valores.

```
a = {"x":1}; b = {"y":2}
a.update(b)
```

Crear una clave únicamente si no existe.

```
freq.setdefault("python", [])
freq["python"].append(10)
```

2.3.4 Eliminar elementos

Eliminar una clave.

```
freq.pop("a")
```

Eliminar devolviendo un valor por defecto.

```
freq.pop("a", 0)
```

Eliminar el último elemento insertado.

```
freq.popitem()
```

Vaciar completamente el diccionario.

```
freq.clear()
```

2.3.5 Funciones integradas

Cantidad de elementos.

```
len(freq)
```

Copiar un diccionario.

```
nuevo = freq.copy()
```

Comprobar existencia.

```
if "python" in freq:
    ...
```

Eliminar una copia sin modificar el original.

```
nuevo = dict(freq)
```

2.3.6 Ordenar un diccionario

Ordenar por clave.

```
sorted(freq.items())
```

Ordenar por valor.

```
sorted(freq.items(), key=lambda kv: kv[1])
```

Orden descendente.

```
sorted(freq.items(), key=lambda kv: -kv[1])
```

2.3.7 defaultdict

Normalmente antes de incrementar una frecuencia se necesita comprobar si la clave existe.

```
freq = {}
freq[x] = freq.get(x, 0) + 1
```

Con defaultdict esto desaparece.

```
from collections import defaultdict

freq = defaultdict(int)

freq[x] += 1
```

También puede utilizarse para listas.

```
grupos = defaultdict(list)
for x in arr:
    grupos[x % 3].append(x)
```

Con conjuntos.

```
adj = defaultdict(set)
adj[u].add(v)
```

Y con cualquier función.

```
from collections import defaultdict
d = defaultdict(lambda: -1)
```

2.3.8 Counter

Cuando únicamente interesa contar frecuencias, la clase Counter simplifica mucho el código.

```
from collections import Counter
freq = Counter(arr)
```

Obtener el elemento más frecuente.

```
freq.most_common(1)
```

Los tres más frecuentes.

```
freq.most_common(3)
```

Frecuencia de un elemento.

```
freq[x]
```

2.3.9 Aplicaciones frecuentes en ICPC

Contar frecuencias.

```
freq = defaultdict(int)
for x in arr:
    freq[x] += 1
```

Agrupar elementos.

```
g = defaultdict(list)
for x in arr:
    g[x % k].append(x)
```

Construir listas de adyacencia.

```
adj = defaultdict(list)
for u, v in edges:
    adj[u].append(v)
```

Construir grafos sin comprobar existencia.

```
adj = defaultdict(set)
adj[u].add(v)
```

Contar caracteres.

```
Counter(s)
```

Resumen

Función	Uso principal
<code>get()</code>	Obtener valor sin <code>KeyError</code>
<code>keys()</code>	Recorrer claves
<code>values()</code>	Recorrer valores
<code>items()</code>	Recorrer clave y valor
<code>update()</code>	Actualizar con otro diccionario
<code>setdefault()</code>	Crear si no existe
<code>pop()</code>	Eliminar una clave
<code>popitem()</code>	Eliminar último elemento
<code>clear()</code>	Vaciar el diccionario
<code>copy()</code>	Copia superficial
<code>len()</code>	Número de elementos
<code>defaultdict</code>	Valor por defecto automático
<code>Counter</code>	Conteo automático de frecuencias

2.4 Manipulación de cadenas

Las cadenas (`str`) son inmutables en Python, por lo que todas las operaciones que modifican su contenido devuelven una nueva cadena.

La mayoría de problemas de parsing, simulación y procesamiento de texto utilizan estas funciones.

2.4.1 `replace()`

Reemplaza ocurrencias de una subcadena.

```
s = "banana"
s.replace("a", "o")
# "bonono"
s.replace("a", "o", 2)
# "bonona"
```

Complejidad promedio: $\mathcal{O}(n)$

2.4.2 `split()`

Divide una cadena utilizando un separador.

```
s.split()
s.split(",")
s.split(":")
s.split("\n")
```

Si no se especifica un separador, utiliza cualquier cantidad de espacios consecutivos.

```
s = "a b c"
print(s.split())
# ['a', 'b', 'c']
```

2.4.3 `join()`

Une una colección de cadenas utilizando un separador.

Es mucho más eficiente que concatenar strings dentro de un ciclo.

```
nums = ["1", "2", "3"]
", ".join(nums)
# "1, 2, 3"
" ".join(nums)
# "1 2 3"
```

Complejidad: $\mathcal{O}(n)$

2.4.4 `strip()`, `lstrip()` y `rstrip()`

Eliminan caracteres al inicio y/o final.

```
s.strip()
s.lstrip()
s.rstrip()
```

También pueden eliminar caracteres específicos.

```
--ICPC--".strip("--")
# ICPC
```

2.4.5 `find()` y `rfind()`

Buscan una subcadena.

Devuelven

- la posición encontrada;
- -1 si no existe.

```
s.find("abc")
s.rfind("abc")
```

2.4.6 `count()`

Cuenta cuántas veces aparece una subcadena.

```
s.count("ab")
```

2.4.7 `startswith()` y `endswith()`

Permiten verificar prefijos y sufijos.

```
s.startswith("abc")
s.endswith(".cpp")
```

Muy útiles para parsing.

2.4.8 `lower()`, `upper()` y `capitalize()`

Cambian el formato de las letras.

```
s.lower()
s.upper()
s.capitalize()
```

2.4.9 `isalpha()`, `isdigit()`, `isalnum()`

Verifican el contenido de una cadena.

```
s.isalpha()
s.isdigit()
s.isalnum()
s.isspace()
```

2.4.10 `partition()`

Divide únicamente en la primera aparición del separador.

```
s.partition("=")
# ('', '=', '15')
```

A diferencia de `split()`, siempre devuelve exactamente tres elementos.

2.4.11 Concatenación

```
a + b
a += b
```

Cuando se concatenan muchas cadenas es preferible utilizar

```
"".join(lista)
```

ya que evita crear múltiples copias intermedias.

2.4.12 Formateo de cadenas

Python permite insertar variables directamente mediante f-strings.

```
x = 3.14159265
f"{x:.2f}"
# 3.14
f"{x:.10f}"
# 3.1415926500
nombre = "ICPC"
f"Hola {nombre}"
```

Este mecanismo es el recomendado para imprimir números reales.

2.4.13 Aplicaciones frecuentes en ICPC

Eliminar espacios al leer una línea.

```
s = input().strip()
```

Separar una línea.

```
tokens = input().split()
```

Unir la respuesta.

```
print(" ".join(ans))
```

Comprobar un prefijo.

```
if s.startswith("YES"):
    ...
```

Eliminar caracteres.

```
s = s.replace(" ", "")
```

Buscar una palabra.

```
if s.find("abc") != -1:
    ...
```

Contar caracteres.

```
vocales = s.count("a")
```

Resumen

Función	Uso principal
<code>replace()</code>	Reemplazar texto
<code>split()</code>	Dividir una cadena
<code>join()</code>	Unir cadenas
<code>strip()</code>	Eliminar espacios
<code>find()</code>	Buscar subcadena
<code>rfind()</code>	Buscar desde el final
<code>count()</code>	Contar ocurrencias
<code>startswith()</code>	Verificar prefijo
<code>endswith()</code>	Verificar sufijo
<code>lower()</code>	Minúsculas
<code>upper()</code>	Mayúsculas
<code>capitalize()</code>	Primera letra mayúscula
<code>isalpha()</code>	Solo letras
<code>isdigit()</code>	Solo dígitos
<code>isalnum()</code>	Alfanumérico
<code>isspace()</code>	Solo espacios
<code>partition()</code>	Dividir una vez

2.5 Módulos matemáticos

Python incluye varios módulos que simplifican operaciones matemáticas frecuentes en programación competitiva.

El más utilizado es `math`.

```
import math
```

2.5.1 Redondeo

Función	Descripción
<code>round(x,n)</code>	Redondea a n decimales
<code>math.floor(x)</code>	Entero menor o igual
<code>math.ceil(x)</code>	Entero mayor o igual
<code>math.trunc(x)</code>	Elimina la parte decimal

```
import math
math.floor(3.8) # 3
math.ceil(3.2) # 4
round(3.141592,2) # 3.14
math.trunc(7.98) # 7
factor = 10 ** N
floor_n = math.floor(x * factor) / factor
ceil_n = math.ceil(x * factor) / factor
```

2.5.2 Valor absoluto

```
abs(x)
math.fabs(x)
```

Normalmente basta utilizar

```
abs(x)
```

2.5.3 Raíces

Función	Uso
<code>math.sqrt(x)</code>	Raíz cuadrada
<code>math.isqrt(x)</code>	Raíz entera exacta

```
math.sqrt(25) # 5.0
math.isqrt(30) # 5
```

En teoría de números suele preferirse `isqrt()`.

2.5.4 Máximo común divisor y mínimo común múltiplo

```
math.gcd(a,b)
math.lcm(a,b)
```

También aceptan más de dos argumentos.

```
math.gcd(a,b,c)
math.lcm(a,b,c)
```

Complejidad: $\mathcal{O}(\log(\min(a, b)))$

2.5.5 Potencias

```
pow(a,b)
pow(a,b,m)
a ** b
math.pow(a,b)
```

Observaciones.

- `pow(a,b,m)` calcula la potencia módulo `m` de forma eficiente.
- `**` devuelve enteros cuando es posible.
- `math.pow()` siempre devuelve float.

2.5.6 Factoriales

```
math.factorial(n)
```

Complejidad: $\mathcal{O}(n)$

2.5.7 Combinatoria

Desde Python 3.8 existen funciones incorporadas.

```
math.comb(n,k)
math.perm(n,k)
```

Ejemplos.

```
math.comb(5,2) # 10
math.perm(5,2) # 20
```

2.5.8 Logaritmos

```
math.log(x)
math.log2(x)
math.log10(x)
```

2.5.9 Trigonometría

```
math.sin(x)
math.cos(x)
math.tan(x)
math.asin(x)
math.acos(x)
```

```
math.atan(x)
```

Los ángulos siempre se expresan en radianes.

2.5.10 Distancias

```
math.dist(p1,p2)
math.hypot(x,y)
```

```
math.hypot(3,4) # 5
```

2.5.11 Producto

```
math.prod(lista)
```

Ejemplo.

```
math.prod([2,3,4]) # 24
```

2.5.12 Constantes

```
math.pi
math.e
math.inf
math.nan
```

2.5.13 Fracciones exactas

Cuando los números en punto flotante producen errores de precisión puede utilizarse el módulo

```
from fractions import Fraction
```

```
Fraction(1,3)
Fraction(1,6)
Fraction(1,3)+Fraction(1,6)
```

Resultado.

```
Fraction(1,2)
```

No existe error de redondeo.

2.5.14 División entera

En Python la división entera siempre redondea hacia menos infinito.

```
7 // 2
# 3
-7 // 2
# -4
```

Este comportamiento es diferente al de C++ y Java.

2.5.15 Aplicaciones frecuentes en ICPC

Máximo común divisor.

```
math.gcd(a,b)
```

Potencia modular.

```
pow(a,b,MOD)
```

Raíz entera.

```
math.isqrt(n)
```

Binomiales.

```
math.comb(n,k)
```

Permutaciones.

```
math.perm(n,k)
```

Fraciones exactas.

```
Fraction(a,b)
```

Resumen

Función	Uso principal
<code>floor()</code>	Redondear hacia abajo
<code>ceil()</code>	Redondear hacia arriba
<code>trunc()</code>	Eliminar parte decimal
<code>gcd()</code>	Máximo común divisor
<code>lcm()</code>	Mínimo común múltiplo
<code>sqrt()</code>	Raíz cuadrada
<code>isqrt()</code>	Raíz entera
<code>factorial()</code>	Factorial
<code>comb()</code>	Combinaciones
<code>perm()</code>	Permutaciones
<code>pow()</code>	Potencias
<code>log2()</code>	Logaritmo base 2
<code>hypot()</code>	Distancia euclidiana
<code>prod()</code>	Producto de un iterable
<code>Fraction</code>	Fraciones exactas

2.6 Librerías estándar útiles para ICPC

Python posee una biblioteca estándar muy completa. Conocer estos módulos permite resolver muchos problemas sin implementar estructuras o algoritmos desde cero.

2.6.1 collections

Contiene estructuras de datos optimizadas.

```
from collections import deque
from collections import defaultdict
from collections import Counter
from collections import OrderedDict
from collections import namedtuple
```

Clase	Uso
<code>deque</code>	Cola doble, BFS, Sliding Window
<code>defaultdict</code>	Dict con valor por defecto
<code>Counter</code>	Conteo automático de frecuencias
<code>OrderedDict</code>	Dict que conserva operaciones de orden
<code>namedtuple</code>	Tuplas con nombres para los campos

a) *deque*

Permite insertar y eliminar elementos por ambos extremos en $\mathcal{O}(1)$

```
from collections import deque
```

```
q = deque()
q.append(5)
q.appendleft(2)
q.pop()
q.popleft()
q.rotate(1)
q.rotate(-1)
```

Es la estructura estándar para implementar BFS.

b) *defaultdict*

Ya estudiado en la sección anterior.

Se utiliza principalmente para:

- frecuencias;
- listas de adyacencia;
- agrupaciones.

c) *Counter*

Muy útil para conteo de frecuencias.

```
freq = Counter(arr)
freq.most_common(3)
```

2.6.2 heapq

Implementa un montículo mínimo (Min Heap).

```
import heapq
```

Funciones principales.

Función	Uso
<code>heappush()</code>	Insertar
<code>heappop()</code>	Extraer mínimo
<code>heapify()</code>	Construir heap
<code>heapreplace()</code>	Reemplazar mínimo
<code>nlargest()</code>	Mayores elementos
<code>nsmallest()</code>	Menores elementos

Ejemplo.

```
heap=[]
heapq.heappush(heap,4)
heapq.heappush(heap,2)
heapq.heappush(heap,8)
heapq.heappop(heap)
```

Todas las operaciones de inserción y extracción tienen complejidad $\mathcal{O}(\log n)$

Se utiliza principalmente en:

- Dijkstra;
- Prim;
- Huffman;
- Top-K.

2.6.3 bisect

Permite realizar búsqueda binaria sobre listas ordenadas.

```
from bisect import bisect_left
from bisect import bisect_right
from bisect import insort
```

```
bisect_left(arr,x)
bisect_right(arr,x)
insort(arr,x)
```

Todas las búsquedas tienen complejidad $\mathcal{O}(\log n)$

2.6.4 functools

Funciones de orden superior.

```
from functools import lru_cache
from functools import cache
from functools import cmp_to_key
from functools import reduce
```

a) *lru_cache*

Muy utilizado para memoización.

```
@lru_cache(None)
def dp(i,j):
    ...
```

b) *cmp_to_key*

Permite utilizar comparadores personalizados.

```
sorted(arr,key=cmp_to_key(cmp))
```

2.6.5 itertools

Herramientas para trabajar con iteradores.

```
from itertools import permutations
from itertools import combinations
from itertools import combinations_with_replacement
from itertools import product
from itertools import accumulate
from itertools import cycle
from itertools import repeat
from itertools import pairwise
```

Función	Uso
permutations	Permutaciones
combinations	Combinaciones
product	Producto cartesiano
accumulate	Prefijos acumulados
cycle	Ciclo infinito
repeat	Repetición
pairwise	Pares consecutivos

Ejemplos.

```
permutations(arr)
combinations(arr,2)
product(a,b)
accumulate(arr)
```

2.6.6 operator

Contiene operadores como funciones.

```
from operator import itemgetter
from operator import attrgetter
```

Ejemplo.

```
sorted(arr,key=itemgetter(1))
```

2.6.7 decimal

Permite realizar aritmética decimal exacta.

```
from decimal import Decimal
```

Normalmente no se utiliza en ICPC debido a su menor rendimiento.

2.6.8 fractions

Permite trabajar con fracciones exactas.

```
from fractions import Fraction
```

Muy útil cuando se desea evitar errores de punto flotante.

2.6.9 array

Implementa arreglos homogéneos.

```
from array import array
```

Consumen menos memoria que una lista convencional.

2.6.10 queue

```
from queue import PriorityQueue
```

Aunque implementa colas de prioridad, en programación competitiva normalmente se prefiere utilizar

```
heapq
```

por ser considerablemente más rápido.

2.6.11 sys

El módulo más utilizado para entrada y salida rápida.

```
import sys

input = sys.stdin.readline

sys.setrecursionlimit(200000)
```

2.6.12 Aplicaciones frecuentes en ICPC

Problema	Librería recomendada
BFS	deque
Dijkstra	heapq
Memoización	lru_cache
Conteo	Counter
Frecuencias	defaultdict
Binary Search	bisect
Permutaciones	itertools
Combinaciones	itertools
Ordenamientos personalizados	cmp_to_key
Prefijos	accumulate

Resumen

Módulo	Uso principal
collections	Estructuras de datos
heapq	Heap mínimo
bisect	Búsqueda binaria
functools	Memoización y comparadores
itertools	Iteradores avanzados
operator	Funciones auxiliares para ordenar
fractions	Fracciones exactas
decimal	Decimales exactos
array	Arreglos homogéneos
queue	Colas sincronizadas
sys	Entrada rápida y recursión

3 Estructuras de Datos

3.1 Fenwick Tree / BIT + Consultas Offline (distintos en un rango)

Preguntas guía:

- ¿Necesito sumas o conteos acumulados sobre un rango que cambia con actualizaciones puntuales?
- ¿Los updates son de un solo índice y las queries son sobre prefijos o rangos $[l, r]$?
- ¿Si no hay updates "online", puedo resolver offline ordenando las queries (p. ej. por su extremo derecho)?

Utilidad: Mantener sumas de prefijo (o cualquier operación invertible) sobre un arreglo con actualizaciones puntuales, respondiendo consultas de rango en tiempo logarítmico. Es la base de técnicas como conteo de elementos distintos por rango, inversiones en un arreglo y frecuencias acumuladas.

Complejidad: Construcción $\mathcal{O}(n)$. Update puntual $\mathcal{O}(\log n)$. Query de prefijo $\mathcal{O}(\log n)$. Espacio $\mathcal{O}(n)$.

Fundamento teórico: Cada posición i del arreglo interno almacena la suma de un bloque de tamaño $(i \& -i)$, el bit menos significativo de i . Al actualizar se "sube" sumando $(i \& -i)$; al consultar un prefijo se "baja" restando $(i \& -i)$. Esto aprovecha la representación binaria de los índices para cubrir cualquier prefijo con solo $\mathcal{O}(\log n)$ bloques.

Ejercicio aplicado

Dado un arreglo de n números y q consultas (i, j) , responder para cada una cuántos valores distintos hay en el subarreglo $[i, j]$. Técnica: procesar las consultas offline ordenadas por extremo derecho r , recorriendo el arreglo de izquierda a derecha y manteniendo en el BIT un "1" solo en la última aparición de cada valor.

I/O: $n \leq 30\,000$ y $q \leq 200\,000$ hasta $\sim 430\,000$ tokens de entrada: se usa lectura total + tokenización (patrón 2) en vez de readline por línea.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.readline
    n = int(data())
    nums = list(map(int, data().split()))
    q = int(data())

    queries = []
    for i in range(q):
        l, r = map(int, data().split())
        queries.append((r, l, i))  # se ordena por r
    queries.sort()

    bit = [0] * (n + 1)

    def update(i, val):
        while i <= n:
            bit[i] += val
            i += i & (-i)  # ultimo bit encendido

    def query(i):
```

```
res = 0
while i > 0:
    res += bit[i]
    i -= i & (-i)
return res

last = {}  # ultima posicion
ans = [0] * q
cur = 0
for r, l, qi in queries:
    while cur < r:  # avanza el puntero hasta r
        cur += 1
        x = nums[cur - 1]
        prev = last.get(x)
        if prev:
            update(prev, -1)  # desactiva la aparicion anterior
        update(cur, 1)  # activa la aparicion actual
        last[x] = cur
    ans[qi] = query(r) - query(l - 1)

sys.stdout.write("\n".join(map(str, ans)))

solve()
```

3.2 Prefix Sums + Mapa de Primera Aparición (subarreglo balanceado más largo)

Preguntas guía:

- ¿Piden la subsecuencia contigua más larga que cumple una condición de "balance" (mismo número de dos tipos de elementos, o suma total igual a un valor fijo)?
- ¿Puedo transformar cada elemento en un delta $(+1/-1/0)$ tal que la condición se traduzca en "suma de un subarreglo == 0"?
- ¿Me alcanza con guardar, para cada valor de suma acumulada, solo la PRIMERA posición donde apareció?

Utilidad: Encontrar en $\mathcal{O}(n)$ el subarreglo contiguo más largo cuya suma (o balance entre dos tipos de elementos) es igual a un valor fijo, sin revisar los $\mathcal{O}(n^2)$ subarreglos posibles.

Complejidad: $\mathcal{O}(n)$ tiempo, $\mathcal{O}(n)$ espacio.

Fundamento teórico: Si $P[i]$ es la suma acumulada de los primeros i elementos (con la transformación adecuada), la suma del subarreglo $(i + 1..j)$ es $P[j] - P[i]$. Esa suma es 0 exactamente cuando $P[j] = P[i]$. Para maximizar la longitud $j - i$, basta con recordar, para cada valor de prefijo visto, la PRIMERA posición en que apareció (nunca la última), y comparar contra ella cada vez que el valor se repite.

Ejercicio aplicado

Dada una secuencia de N transacciones (positiva = depósito, negativa = retiro, cero = consulta), calcular el depósito máximo, el retiro más negativo, y la longitud del subperíodo contiguo más largo con igual cantidad de depósitos y retiros.

I/O: Múltiples casos con centinela "0" (patrón 3). $N \leq 10\,000 \rightarrow$ readline por línea (patrón 1); se probó también

con lectura total y resultó más lenta para este patrón de "un entero, luego un arreglo, repetido" (~2.5s vs ~3.9s en 500 casos de $N=10000$).

Código solución

```
import sys
input = sys.stdin.buffer.readline

def solve():
    out = []
    while True:
        n = int(input())
        if n == 0:
            break
        arr = list(map(int, input().split()))

        d = 0
        w = 0
        for x in arr:
            if x > 0:
                d = max(d, x)
            elif x < 0:
                w = min(w, x) # el mas negativo, no
                               # el de mayor magnitud

        # prefijo: +1 por deposito, -1 por retiro, 0 no
        # cambia nada
        first_seen = {0: 0} # valor de prefijo ->
        PRIMERA posicion vista
        prefix = 0
        r = 0
        for i, x in enumerate(arr, start=1):
            if x > 0:
                prefix += 1
            elif x < 0:
                prefix -= 1
            if prefix in first_seen:
                r = max(r, i - first_seen[prefix]) # se
            repite: hay balance
            else:
                first_seen[prefix] = i #
                primera vez: se guarda

        out.append(f"{d} {w} {r}")
        sys.stdout.write("\n".join(out) + "\n")
    solve()
```

3.3 Fenwick Tree con XOR + Sprague-Grundy (Nim por rangos)

Preguntas guía:

- ¿El juego es una suma de sub-juegos independientes (aquí: columnas de fragmentos) donde en cada turno se modifica UNA sola pila?
- ¿Necesito responder "quién gana" para muchos rangos distintos, con actualizaciones puntuales intercaladas?
- ¿La operación de combinación relevante es XOR en vez de suma?

Utilidad: Generalizar el Fenwick Tree (visto para sumas) a CUALQUIER operación invertible para mantener el valor de Grundy combinado de un rango de pilas de Nim, con actualizaciones puntuales en tiempo logarítmico. Combinado con el teorema de Sprague-Grundy/Bouton, responde instantáneamente quién gana un juego de Nim sobre cualquier subrango.

Complejidad: Construcción $\mathcal{O}(n)$. Update puntual $\mathcal{O}(\log n)$. Query de rango $\mathcal{O}(\log n)$.

Fundamento teórico: En Nim con pilas independientes donde un jugador retira cualquier cantidad positiva de UNA pila, el valor de Grundy de una pila de tamaño v es exactamente v . Por el teorema de Bouton, el jugador que mueve PRIMERO gana si y solo si el XOR de todas las pilas en juego es distinto de 0. Como XOR es su propio inverso ($v \oplus v = 0$), el mismo esquema del Fenwick Tree que acumula sumas de prefijo funciona igual acumulando XOR de prefijo: para modificar el valor de una posición de v_{viejo} a v_{nuevo} , basta aplicar un update con delta $= v_{viejo} \oplus v_{nuevo}$ (equivalente a "quitar" el valor viejo y "poner" el nuevo en una sola operación).

Ejercicio aplicado

Juan y Frank juegan Nim sobre columnas de fragmentos: en cada turno, un jugador retira cualquier cantidad positiva de fragmentos de UNA columna; pierde quien no puede mover. Dadas Q consultas, cada una una "profecía" (determinar el ganador en un rango $[l, r]$, sabiendo que Frank siempre empieza) o un "ritual" (sumar x fragmentos a una columna), resolver cada profecía.

I/O: $N, Q \leq 10^6 \rightarrow$ lectura total con tokenización (patrón 2), y las funciones de update/query del Fenwick se escriben SIN llamadas a función (inline) para reducir el overhead de Python en este volumen de operaciones. Aun así, en el peor caso ($N=Q=10^6$) la solución pura en Python ronda ~7s si el juez es muy estricto con el límite de tiempo, este es el techo práctico de Python puro para este volumen sin cambiar de lenguaje.

Código solución

```
import sys

def main():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); q = int(data[idx + 1]); idx += 2
    arr = [0] * (n + 1)
    tree = [0] * (n + 1)

    # construccion en O(n): cada nodo se combina UNA vez
    # con su padre directo,
    # en vez de hacer n updates completos de O(log n)
    # cada uno
    for i in range(1, n + 1):
        v = int(data[idx]); idx += 1
        arr[i] = v
        tree[i] ^= v
        j = i + (i & (-i))
        if j <= n:
            tree[j] ^= tree[i]

    out = []
    out_append = out.append
    ANS = ("JUAN", "FRANK")
    P_BYTE = 80 # ord('P'): comparar el primer byte
                 # evita decodificar el token

    i2 = idx
    for _ in range(q):
        tok = data[i2]
        if tok[0] == P_BYTE:
```

```

    l = int(data[i2 + 1]); r = int(data[i2 + 2]);
    i2 += 3
    # xor(prefijo r) ^ xor(prefijo l-1) = xor del
    rango [l, r]
    res = 0
    i = r
    while i:
        res ^= tree[i]
        i &= i - 1      # equivalente a i -=
i & (-i), un poco mas rapido
        i = l - 1
        while i:
            res ^= tree[i]
            i &= i - 1
            out_append(ANS[res != 0])  # XOR != 0 ->
gana quien mueve primero (Frank)
        else:
            k = int(data[i2 + 1]); x = int(data[i2 + 2]);
            i2 += 3
            old = arr[k]
            new = old + x
            delta = old ^ new          # "sacar" old y "
            poner" new en un solo update
            arr[k] = new
            i = k
            while i <= n:
                tree[i] ^= delta
                i += i & (-i)

    sys.stdout.write("\n".join(out) + "\n")

main()

```

4 Algoritmos de Grafos

4.1 Emparejamiento Bipartito Máximo, Kuhn (parejas de baile)

Preguntas guía:

- ¿Tengo dos conjuntos disjuntos (personas / tareas) y aristas que solo conectan un conjunto con el otro?
- ¿Me piden el número máximo de "parejas" o asignaciones simultáneas sin repetir elementos de ningún lado?
- ¿El tamaño de ambos lados permite $\mathcal{O}(V \cdot E)$ (hasta unos pocos miles)?

Utilidad: Encontrar el máximo número de aristas seleccionables en un grafo bipartito sin que dos aristas compartan vértice (asignación óptima uno a uno). Ejemplos: asignar trabajadores a tareas, parejas de baile, cobertura de tareas.

Complejidad: $\mathcal{O}(V \cdot E)$ con la versión simple de caminos aumentantes, donde V es el lado izquierdo y E el número de aristas.

Fundamento teórico: Se buscan repetidamente "caminos aumentantes": para cada vértice izquierdo se intenta, vía DFS, encontrar una pareja libre o "desalojar" a otro vértice ya emparejado para liberar espacio y aumentar el emparejamiento en 1. Es un caso particular del teorema de König / flujo máximo en redes.

Ejercicio aplicado

Hay n chicos y m chicas y k parejas de baile posibles. Hallar el número máximo de parejas simultáneas y mostrar una asignación válida.

I/O: $n, m \leq 500$ y $k \leq 1000 \rightarrow$ entrada pequeña, se usa readline por línea (patrón 1), suficiente y más simple de leer.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def try_kuhn(u, adj, matched, visited):
    # intenta emparejar el vertice u del lado izquierdo
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            # v esta libre, o su pareja actual puede
            # reubicarse...
            if matched[v] == -1 or try_kuhn(matched[v],
            adj, matched, visited):
                matched[v] = u
                return True
    return False

def solve():
    n, m, k = map(int, input().split())
    adj = [[] for _ in range(n)]
    for _ in range(k):
        a, b = map(int, input().split())
        adj[a - 1].append(b - 1)

    matched = [-1] * m                # pareja actual
    # de cada vertice derecho
    total = 0
    for u in range(n):
        visited = [False] * m         # se reinicia en
        # cada intento
        if try_kuhn(u, adj, matched, visited):
            total += 1
```

```
def try_kuhn(u, adj, matched, visited):
    total += 1

    out = [str(total)]
    for v in range(m):
        if matched[v] != -1:
            out.append(f"{matched[v] + 1} {v + 1}")
    sys.stdout.write("\n".join(out))

solve()
```

4.2 Kuhn sobre Segmentos de Fila/Columna (torres con peones)

Preguntas guía:

- ¿La cuadrícula tiene obstáculos que "cortan" el alcance de piezas o rangos dentro de cada fila y columna?
- ¿Cada fila y cada columna se puede dividir en segmentos independientes (separados por obstáculos) donde a lo más un elemento puede colocarse por segmento?
- ¿Piden el máximo número de elementos colocables sin conflicto entre sí?

Utilidad: Resolver problemas de colocación de piezas sobre una cuadrícula con obstáculos, donde el conflicto es "compartir fila o columna sin un obstáculo de por medio". En vez de razonar celda por celda, se etiquetan los segmentos libres de cada fila y columna, y el problema se convierte en un emparejamiento bipartito clásico.

Complejidad: $\mathcal{O}(N^2)$ para etiquetar segmentos, más $\mathcal{O}(V \cdot E)$ para el emparejamiento (V, E acotados por N^2).

Fundamento teórico: Cada casilla vacía pertenece a exactamente un segmento de fila (corrida maximal de casillas vacías, delimitada por obstáculos o bordes) y a exactamente un segmento de columna. Como a lo más una torre puede ocupar cada segmento (todas sus casillas se atacan entre sí sin obstáculo de por medio), colocar torres sin conflicto equivale a elegir aristas entre segmentos de fila y segmentos de columna sin repetir segmento de ningún lado: exactamente un emparejamiento en el grafo bipartito (segmentos de fila) $\times$ (segmentos de columna), donde cada casilla vacía es una arista posible.

Ejercicio aplicado

Dado un tablero $N \times N$ con algunas casillas ocupadas por peones, hallar el máximo número de torres que se pueden colocar en casillas vacías sin que se ataquen entre sí (dos torres se atacan si comparten fila o columna sin un peón entre ellas). La entrada trae varios tableros seguidos.

I/O: $N \leq 100$, múltiples casos hasta EOF (patrón 4 de la sección de E/S) $\rightarrow$ lectura total con `sys.stdin.read().split()` y procesamiento por índice.

Código solución

```
import sys

def try_kuhn(u, adj, matched, visited):
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            if matched[v] == -1 or try_kuhn(matched[v],
            adj, matched, visited):
                matched[v] = u
                return True
    return False
```

```

        matched[v] = u
        return True
    return False

def solve_board(n, grid):
    # etiqueta cada corrida maximal de celdas vacias en
    # cada FILA con un id
    row_id = [[-1] * n for _ in range(n)]
    row_count = 0
    for r in range(n):
        c = 0
        while c < n:
            if grid[r][c] == '.':
                while c < n and grid[r][c] == '.':
                    row_id[r][c] = row_count
                    c += 1
                row_count += 1
            else:
                c += 1

    # lo mismo por COLUMNAS
    col_id = [[-1] * n for _ in range(n)]
    col_count = 0
    for c in range(n):
        r = 0
        while r < n:
            if grid[r][c] == '.':
                while r < n and grid[r][c] == '.':
                    col_id[r][c] = col_count
                    r += 1
                col_count += 1
            else:
                r += 1

    # grafo bipartito: segmentos de fila (izq) --
    # segmentos de columna (der);
    # cada celda vacia es una arista posible entre ambos
    # segmentos
    adj = [[] for _ in range(row_count)]
    for r in range(n):
        for c in range(n):
            if grid[r][c] == '.':
                adj[row_id[r][c]].append(col_id[r][c])

    matched = [-1] * col_count
    total = 0
    for u in range(row_count):
        visited = [False] * col_count
        if try_kuhn(u, adj, matched, visited):
            total += 1
    return total

def main():
    data = sys.stdin.read().split()
    idx = 0
    out = []
    while idx < len(data):
        n = int(data[idx]); idx += 1
        grid = data[idx:idx + n]; idx += n
        out.append(str(solve_board(n, grid)))
    sys.stdout.write("\n".join(out) + "\n")

main()

```

4.3 Teorema de König: Cobertura Mínima de Vértices (quitar caballos)

Preguntas guía:

- ¿Piden el mínimo número de elementos a eliminar para que ningún conflicto (arista) quede activo?

- ¿El grafo de conflictos es bipartito (se puede partir en 2 grupos donde todo conflicto va de un grupo al otro)?
- ¿Ya sé calcular el emparejamiento máximo de ese grafo (entrada anterior, Kuhn)?

Utilidad: Calcular la cobertura mínima de vértices (mínimo conjunto de nodos que toca todas las aristas) en un grafo bipartito. En grafos generales este problema es NP-difícil, pero en grafos bipartitos se vuelve polinomial gracias al teorema de König.

Complejidad: $\mathcal{O}(V \cdot E)$, la misma del emparejamiento bipartito máximo del que depende.

Fundamento teórico: El teorema de König establece que, en cualquier grafo bipartito, el tamaño de la cobertura mínima de vértices es EXACTAMENTE igual al tamaño del emparejamiento máximo. Basta con correr Kuhn (entrada anterior) y esa cifra ya es la respuesta. Importante: heurísticas locales (como quitar iterativamente puntos de articulación, o dividir el número de vértices con grado > 0 entre 2) NO son válidas para este problema en general - solo coinciden por casualidad en casos muy simples (aristas aisladas, caminos, estrellas) y fallan en cuanto el grafo tiene una estructura más rica (por ejemplo, un grafo bipartito 2-conexo sin puntos de articulación).

Ejercicio aplicado

Dado un tablero $N \times N$ con K caballos de ajedrez colocados, hallar el mínimo número de caballos que hay que quitar para que ninguno ataque a otro. El grafo de ataques es bipartito por construcción: cada movimiento de caballo cambia la paridad de (fila + columna), así que alcanza con particionar por esa paridad y correr Kuhn.

I/O: $N \leq 25$, $K \leq N^2 \leq 625 \rightarrow$ entrada pequeña, readline por línea (patrón 1).

Código solución

```

import sys
input = sys.stdin.buffer.readline

def try_kuhn(u, adj, matched, visited):
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            if matched[v] == -1 or try_kuhn(matched[v],
            adj, matched, visited):
                matched[v] = u
                return True
    return False

def max_bipartite_matching(left_nodes, n_right, adj):
    matched = [-1] * n_right
    total = 0
    for u in left_nodes:
        visited = [False] * n_right
        if try_kuhn(u, adj, matched, visited):
            total += 1
    return total

def solve():
    n, k = map(int, input().split())
    pos = []
    for _ in range(k):
        r, c = map(int, input().split())
        pos.append((r - 1, c - 1))

```

```

index = {p: i for i, p in enumerate(pos)}
moves = [(1,2),(2,1),(-1,2),(-2,1),(1,-2),(2,-1),
        ,(-1,-2),(-2,-1)]

# el ataque de caballo SIEMPRE cambia el color (fila+
# col) par/impar:
# el grafo de ataques es bipartito por construccion,
# sin necesidad
# de demostrarlo -- solo hay que particionar por ese
# color
left_nodes = [i for i, (r, c) in enumerate(pos) if (r
+ c) % 2 == 0]
adj = [[] for _ in range(k)]
for i in left_nodes:
    r, c = pos[i]
    for dr, dc in moves:
        j = index.get((r + dr, c + dc))
        if j is not None:
            adj[i].append(j)

# Konig: cobertura minima de vertices ==
# emparejamiento maximo
print(max_bipartite_matching(left_nodes, k, adj))

solve()

```

4.4 Componentes Fuertemente Conexas, Kosaraju (reinos de planetas)

Preguntas guía:

- ¿Es un grafo dirigido y preguntan por "grupos" donde todo par de nodos es mutuamente alcanzable?
- ¿Necesito "condensar" el grafo (contraer cada componente en un nodo) para trabajar luego sobre un DAG?
- ¿n puede ser grande ($\sim 10^5$), por lo que hay que evitar recursión profunda en Python?

Utilidad: Particionar un grafo dirigido en componentes fuertemente conexas (SCC): subconjuntos maximales donde cada nodo puede llegar a cualquier otro del mismo subconjunto. Es la base para condensar grafos, resolver 2-SAT y detectar ciclos mutuos.

Complejidad: $O(V + E)$ tiempo y espacio (dos recorridos DFS).

Fundamento teórico: Kosaraju usa dos pasadas de DFS: (1) en el grafo original, registrar el orden de finalización (postorden) de cada nodo; (2) en el grafo transpuesto (aristas invertidas), recorrer los nodos en orden inverso de finalización. Cada árbol de DFS resultante en esta segunda pasada es exactamente una SCC, porque invertir las aristas separa los componentes que antes solo eran alcanzables en una dirección.

Ejercicio aplicado

Un juego tiene n planetas conectados por m teletransportadores dirigidos. Dos planetas pertenecen al mismo "reino" si hay ruta en ambos sentidos entre ellos. Determinar, para cada planeta, a qué reino pertenece.

I/O: $n \leq 10^5$ y $m \leq 2 \cdot 10^5 \rightarrow$ lectura total + tokenización (patrón 2). Además, el DFS se implementa de forma iterativa (con pila explícita) para evitar RecursionError en Python con grafos grandes.

Código solución

```

import sys

def kosaraju(n, adj):
    visited = [False] * n
    order = [] # orden de finalizacion (
    postorden)
    for s in range(n):
        if visited[s]:
            continue
        visited[s] = True
        stack = [(s, iter(adj[s]))] # iterador
        # recuerda donde va en i parte de la lista
        while stack:
            node, it = stack[-1]
            advanced = False
            for nxt in it: # DFS iterativo
                con iteradores
                if not visited[nxt]:
                    visited[nxt] = True
                    stack.append((nxt, iter(adj[nxt])))
                    advanced = True
                    break
            if not advanced: # no quedan
                vecinos: cierra el nodo
                order.append(node)
                stack.pop()

    radj = [[] for _ in range(n)] # grafo
    # transpuesto (aristas invertidas)
    for u in range(n):
        for v in adj[u]:
            radj[v].append(u)

    comp = [0] * n
    visited = [False] * n
    c = 0
    for node in reversed(order): # orden inverso
        de finalizacion
        if visited[node]:
            continue
        c += 1
        visited[node] = True
        comp[node] = c
        stack = [node]
        while stack:
            u = stack.pop()
            for v in radj[u]:
                if not visited[v]:
                    visited[v] = True
                    comp[v] = c
                    stack.append(v)

    return c, comp

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); m = int(data[idx + 1]); idx += 2
    adj = [[] for _ in range(n)]
    for _ in range(m):
        a = int(data[idx]); b = int(data[idx + 1]); idx
        += 2
        adj[a - 1].append(b - 1)

    k, comp = kosaraju(n, adj)
    out = [str(k), " ".join(map(str, comp))]
    sys.stdout.write("\n".join(out))

solve()

```

4.5 Circuito Euleriano, Hierholzer (ruta postal)

Preguntas guía:

- ¿Piden recorrer TODAS las aristas exactamente una vez (no todos los nodos, como en Hamiltoniano)?
- ¿Se pide un camino o circuito que use cada calle/arista una sola vez?
- ¿Puedo verificar rápido la condición de existencia (grados pares para circuito; 0 ó 2 nodos de grado impar para camino) antes de construir la ruta?

Utilidad: Encontrar un recorrido que use cada arista del grafo exactamente una vez (circuito o camino euleriano). Aplicaciones: rutas de reparto/barrido que cubren todas las calles, o cualquier problema de "recorrer cada arista una vez".

Complejidad: $\mathcal{O}(V + E)$ con la versión iterativa basada en pila (evita recursión y eliminaciones costosas de aristas).

Fundamento teórico: Existe circuito euleriano si y solo si el grafo es conexo (en aristas) y todos los vértices tienen grado par. El algoritmo de Hierholzer avanza por aristas no usadas hasta atascarse; en ese momento inserta el vértice atascado en la ruta final y retrocede. El resultado, leído en el orden en que se van "atascando" los vértices, es un circuito válido.

Ejercicio aplicado

Dado un mapa de n cruces y m calles bidireccionales, hallar una ruta que empiece y termine en el cruce 1 (oficina postal) y recorra cada calle exactamente una vez, o determinar que es "IMPOSSIBLE".

I/O: $n \leq 10^5$, $m \leq 2 \cdot 10^5 \rightarrow$ lectura total + tokenización (patrón 2). Se evita `adj.erase()/remove()` ($\mathcal{O}(n)$ por operación) usando un arreglo de punteros `ptr[]` que salta aristas usadas: así la eliminación es $\mathcal{O}(1)$ amortizado.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); m = int(data[idx + 1]); idx += 2

    adj = [[] for _ in range(n + 1)]
    deg = [0] * (n + 1)
    for e in range(m):
        a = int(data[idx]); b = int(data[idx + 1]); idx += 2
        adj[a].append((b, e))
        adj[b].append((a, e))          # grafo no dirigido:
        # ambos sentidos (e es id de arista)
        deg[a] += 1
        deg[b] += 1

    if deg[1] == 0 or any(deg[v] % 2 for v in range(1, n + 1)):
        print("IMPOSSIBLE")
        return

    used = [False] * m
    ptr = [0] * (n + 1)              # guarda las ids de
    # las aristas

    path = []
    stack = [1]
    while stack:
        v = stack[-1]
        while ptr[v] < len(adj[v]) and used[adj[v][ptr[v]][1]]:
            ptr[v] += 1
            # salta aristas
            # ya usadas (O(1) amortizado)
        if ptr[v] == len(adj[v]):
            # reviso todas
            # las aristas de v y no queda ninguna disponible
            path.append(v)
            stack.pop()
        else:
            u, eid = adj[v][ptr[v]]
            used[eid] = True
            stack.append(u)
            # avanza por la
            # arista disponible

    if len(path) - 1 != m:
        # no se
        # recorrieron todas las aristas
        print("IMPOSSIBLE")
        return

    sys.stdout.write(" ".join(map(str, path)))

solve()
```

```
path = []
stack = [1]
while stack:
    v = stack[-1]
    while ptr[v] < len(adj[v]) and used[adj[v][ptr[v]][1]]:
        ptr[v] += 1
        # salta aristas
        # ya usadas (O(1) amortizado)
    if ptr[v] == len(adj[v]):
        # reviso todas
        # las aristas de v y no queda ninguna disponible
        path.append(v)
        stack.pop()
    else:
        u, eid = adj[v][ptr[v]]
        used[eid] = True
        stack.append(u)
        # avanza por la
        # arista disponible

if len(path) - 1 != m:
    # no se
    # recorrieron todas las aristas
    print("IMPOSSIBLE")
    return

sys.stdout.write(" ".join(map(str, path)))

solve()
```

4.6 Puntos de Articulación, Tarjan (islas críticas)

Preguntas guía:

- ¿Piden identificar nodos cuya eliminación desconecta el grafo (o aumenta el número de componentes)?
- ¿El grafo es no dirigido y se pregunta por robustez/conectividad ante fallos de un solo nodo?
- ¿ n puede ser grande, por lo que hay que evitar recursión profunda en Python?

Utilidad: Encontrar los vértices de corte (articulation points) de un grafo no dirigido: nodos que, al eliminarse, aumentan el número de componentes conexas. Es la base para analizar robustez de redes (qué nodo, si falla, parte el sistema en pedazos).

Complejidad: $\mathcal{O}(V + E)$ con DFS iterativo (un solo recorrido con low-link).

Fundamento teórico: Se hace un DFS calculando, para cada nodo, su tiempo de descubrimiento (`disc`) y el menor tiempo de descubrimiento alcanzable usando a lo más una arista de retroceso (`low`). Un nodo u (no raíz) es punto de articulación si tiene un hijo v en el árbol DFS tal que $\text{low}[v] \geq \text{disc}[u]$: eso significa que desde el subárbol de v no hay forma de "escapar" hacia arriba de u sin pasar por u . La raíz del DFS es un caso especial: es punto de articulación si tiene más de un hijo en el árbol.

Ejercicio aplicado

Dadas varias pruebas, cada una con n islas y m puentes bidireccionales (grafo conexo), contar cuántas islas, al hundirse, desconectarían partes de la ciudad. La entrada termina con un caso 0 0.

I/O: Múltiples casos con centinela "0 0" (patrón 3 de la sección de E/S). $n \leq 10^4$, $m \leq 10^5 \rightarrow$ lectura total + toke-

nización, y DFS iterativo con pila explícita (mismo motivo que en Kosaraju: evitar RecursionError/stack overflow).

Código solución

```
import sys

def find_articulation_points(n, adj):
    disc = [0] * n
    low = [0] * n
    visited = [False] * n
    is_ap = [False] * n
    timer = 1

    for start in range(n):
        if visited[start]:
            continue
        visited[start] = True
        disc[start] = low[start] = timer
        timer += 1
        root_children = 0
        # pila: [nodo, padre, iterador de vecinos,
        # ya_salto_arista_al_padre]
        stack = [[start, -1, iter(adj[start]), False]]
        while stack:
            frame = stack[-1]
            node, parent, it, _ = frame
            advanced = False
            for nxt in it:
                if nxt == parent and not frame[3]:
                    frame[3] = True # salta SOLO una
                    # vez la arista de vuelta
                    continue # (por si hay
                    # aristas paralelas)
                if not visited[nxt]:
                    visited[nxt] = True
                    disc[nxt] = low[nxt] = timer
                    timer += 1
                    if node == start:
                        root_children += 1
                    stack.append([nxt, node, iter(adj[nxt]), False])
                    advanced = True
                    break
            else:
                low[node] = min(low[node], disc[nxt])
                # arista de retroceso
                if not advanced:
                    stack.pop()
                if stack:
                    par = stack[-1]
                    par_node = par[0]
                    low[par_node] = min(low[par_node],
                    low[node])
                    # condicion clasica de punto de
                    # articulation (no raiz)
                    if par_node != start and low[node] >=
                    disc[par_node]:
                        is_ap[par_node] = True
                    if root_children > 1: # caso especial: la
                    # raiz del DFS
                        is_ap[start] = True

        return is_ap

def main():
    data = sys.stdin.buffer.read().split()
    idx = 0
    out = []
    while True:
        n = int(data[idx]); m = int(data[idx + 1]); idx
        += 2
        if n == 0 and m == 0:
            break
```

```
adj = [[] for _ in range(n)]
for _ in range(m):
    u = int(data[idx]) - 1; v = int(data[idx +
1]) - 1; idx += 2
    adj[u].append(v)
    adj[v].append(u)
is_ap = find_articulation_points(n, adj)
out.append(str(sum(is_ap)))
sys.stdout.write("\n".join(out) + "\n")

main()
```

4.7 Bron–Kerbosch: Clique Máximo (grupo compatible)

Preguntas guía:

- ¿Necesito encontrar un clique máximo o todos los cliques máximos de un grafo no dirigido?
- ¿El número de vértices es relativamente pequeño (aproximadamente $n \leq 50$)?
- ¿La solución requiere explorar todas las posibles extensiones de un conjunto de vértices compatible?

Utilidad: Encontrar todos los cliques máximos o el clique máximo de un grafo no dirigido mediante backtracking con poda. Es la técnica estándar para resolver problemas de máxima compatibilidad, grupos completamente conectados y optimización sobre grafos pequeños.

Complejidad: $\mathcal{O}(3^{n/3})$ en el peor caso, que es óptimo para la enumeración de todos los cliques máximos; memoria $\mathcal{O}(n)$. En la práctica, el uso de pivote reduce considerablemente el número de llamadas recursivas.

Fundamento teórico: Durante la búsqueda se mantienen tres conjuntos:

- R : vértices que pertenecen al clique actual.
- P : vértices que todavía pueden añadirse al clique.
- X : vértices ya procesados, utilizados para evitar generar soluciones repetidas.

Cuando tanto P como X quedan vacíos, el conjunto R constituye un clique máximo. Para reducir la cantidad de ramas exploradas se selecciona un pivote u y únicamente se consideran los vértices de $P \setminus N(u)$, donde $N(u)$ representa los vecinos del pivote. Esta optimización disminuye significativamente el árbol de búsqueda.

Ejercicio aplicado

Dado un grafo no dirigido, determinar el tamaño del mayor clique del grafo. Cada vértice representa una persona y existe una arista cuando dos personas son compatibles. Se desea encontrar el grupo más grande donde todos los integrantes sean compatibles entre sí.

I/O: Se reciben múltiples casos de prueba; cada caso especifica el número de vértices y aristas seguido por la lista de conexiones. El algoritmo resulta adecuado para grafos pequeños, donde una búsqueda exhaustiva con poda es suficiente para obtener la solución óptima.

Código solución

```
def bronk(R, P, X):
    global best
```

```

if not P and not X:
    best = max(best, len(R))
    return

if P or X:
    u = next(iter(P | X))
else:
    u = None

if u is None:
    candidates = list(P)
else:
    candidates = list(P - graph[u])

for v in candidates:
    bronk(
        R | {v},
        P & graph[v],
        X & graph[v]
    )

    P.remove(v)
    X.add(v)

n, m = map(int, input().split())
graph = [set() for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    graph[u].add(v)
    graph[v].add(u)

best = 0
bronk(set(), set(range(n)), set())
print(best)

```

Observación: El algoritmo de Bron–Kerbosch explora únicamente conjuntos que pueden convertirse en un clique válido. La versión con pivote es la utilizada habitualmente en programación competitiva debido a que reduce considerablemente el número de llamadas recursivas. Aunque su complejidad en el peor caso sigue siendo exponencial, resulta extremadamente eficiente para grafos pequeños, siendo la solución estándar para problemas de clique máximo en ICPC.

4.8 Orden Topológico, Kahn + Propagación de Conteos (Digi Comp II)

Preguntas guía:

- ¿Tengo un DAG donde una cantidad enorme de "elementos" entra por un único origen y debo saber cuántos pasan por cada nodo, siendo imposible simularlos uno por uno?
- ¿La forma en que un nodo reparte su cantidad entre sus sucesores depende solo del TOTAL que lo atraviesa (y no del orden individual de llegada)?
- ¿Necesito procesar cada nodo únicamente cuando TODOS sus predecesores ya aportaron su parte, es decir, en orden topológico?
- ¿El resultado final por nodo depende solo de la paridad o de un residuo del total acumulado?

Utilidad: Sustituye la simulación individual de n recorridos sobre un DAG (inviabile cuando n es astronómico) por un único barrido en orden topológico que propaga CONTEOS agregados en lugar de elementos. El orden

topológico de Kahn garantiza que, al procesar un nodo, ya se conoce su total definitivo, de modo que el reparto hacia sus sucesores se calcula una sola vez, en tiempo lineal, sin importar cuán grande sea n .

Complejidad: $\mathcal{O}(V + E)$ para el orden topológico y la propagación, con $E = 2V$ aquí (cada nodo tiene exactamente dos aristas de salida).

Fundamento teórico: La observación clave es que el efecto de n bolas sobre un interruptor NO depende del orden en que llegan: si t bolas pasan por un vértice v , el interruptor alterna su estado en cada una, así que las bolas se reparten alternadamente entre las dos salidas. Si el estado inicial es L, salen $\lceil t/2 \rceil$ por la izquierda y $\lfloor t/2 \rfloor$ por la derecha (y al revés si el estado inicial es R); el estado FINAL del interruptor solo depende de la PARIDAD de t : se invierte si t es impar y queda igual si es par. Esto convierte un problema aparentemente secuencial en uno puramente aritmético. Para que el cálculo sea válido hace falta conocer el total t de cada vértice ANTES de procesarlo, lo cual es exactamente lo que garantiza el orden topológico: el algoritmo de Kahn mantiene un contador de grado de entrada y solo encola un vértice cuando todas sus aristas entrantes han sido consumidas, es decir, cuando todos sus predecesores ya sumaron su aporte a `count`. El enunciado garantiza que el grafo es acíclico (una bola siempre baja y nunca puede volver a un interruptor), condición necesaria para que exista dicho orden. Nótese que cuando $L = R$ el vértice destino recibe DOS unidades de grado de entrada y también se decrementa dos veces, por lo que el conteo sigue siendo consistente y el vértice no se encola prematuramente.

Ejercicio aplicado

Una máquina Digi Comp II se modela como un DAG de m interruptores, cada uno con estado L/R y dos aristas de salida. Entran n bolas por el vértice 1; cada bola sigue la salida indicada por el estado del interruptor y lo invierte al pasar. Reportar el estado final de los m interruptores.

I/O: $m \leq 5 \cdot 10^5$ con 3 tokens por interruptor (carácter + dos enteros) $\Rightarrow$ lectura total + tokenización con índice `idx` (patrón 2), comparando el carácter directamente como `bytes` (`b'L'`) sin decodificar. $n \leq 10^{18}$ cabe sin problema en los enteros nativos de Python. La salida se arma en un `bytearray` de m posiciones escribiendo los códigos ASCII 76/82 (`'L'/'R'`) y se emite con un único `sys.stdout.buffer.write`, evitando concatenar medio millón de strings. Verificado contra el caso de ejemplo y contra una simulación bola por bola en 500 DAGs aleatorios pequeños (incluyendo casos con $L = R$) sin discrepancias, más casos borde ($n = 0$, $m = 1$, cadena de profundidad máxima, todos los nodos apuntando al vértice final). En el peor caso ($m = 500\,000$, $n = 10^{18}$) se midieron ~ 1.43 s en una cadena de profundidad máxima y ~ 0.97 s en un DAG denso, frente a un límite de 7 segundos.

Código solución

```

import sys
from collections import deque

```

```

data = sys.stdin.buffer.read().split()
idx = 0
n = int(data[idx])
m = int(data[idx+1])
idx += 2

gamestate = [0]*(m+1)    # 1 = L, 2 = R
left = [0]*(m+1)
right = [0]*(m+1)
indegree = [0]*(m+1)

for i in range(1, m+1):
    c = data[idx]
    L = int(data[idx+1])
    R = int(data[idx+2])
    idx += 3
    gamestate[i] = 1 if c == b'L' else 2
    left[i] = L
    right[i] = R
    indegree[L] += 1      # si L == R, el destino suma 2 (
    y luego se resta 2)
    indegree[R] += 1

count = [0]*(m+1)        # cuantas bolas pasan por cada
                           vertice
count[1] = n              # todas las bolas entran por el
                           vertice 1
final_state = gamestate[:]

queue = deque(v for v in range(m+1) if indegree[v]==0)

while queue:
    v = queue.popleft()
    if v == 0:
        continue          # el vertice 0 es el final: no
                           reparte nada

    total = count[v]
    half = total//2
    odd = total&1
    # el interruptor alterna: el reparto es mitad y mitad
    # y la bola
    # sobrante (si total es impar) se va por el lado del
    # estado INICIAL
    if gamestate[v] == 1:
        left_count = half+odd
        right_count = half
    else:
        right_count = half+odd
        left_count = half
    if odd:
        # solo la paridad del total determina si el
        # estado queda invertido
        final_state[v] = 2 if gamestate[v] == 1 else 1

    lv, rv = left[v], right[v]
    count[lv] += left_count
    indegree[lv] -= 1
    if indegree[lv] == 0:    # ya recibio el aporte de
        TODOS sus predecesores
        queue.append(lv)
    count[rv] += right_count
    indegree[rv] -= 1
    if indegree[rv] == 0:
        queue.append(rv)

out = bytearray(m)
for i in range(1, m+1):
    out[i-1] = 76 if final_state[i] == 1 else 82    # 76='
    L', 82='R'
sys.stdout.buffer.write(bytes(out))

```

4.9 Union-Find / DSU con Path Halving (componentes conexas)

Preguntas guía:

- ¿Me piden cuántos grupos/islas/componentes separados hay en un grafo no dirigido, o si dos elementos están en el mismo grupo?
- ¿Las aristas llegan de forma incremental (una por una) y solo necesito consultar conectividad, sin nunca eliminar aristas?
- ¿Puedo tolerar aristas repetidas y lazos sin que el conteo se rompa?
- ¿Prefiero evitar construir listas de adyacencia y hacer DFS/BFS explícito?

Utilidad: Mantiene una partición de elementos en conjuntos disjuntos soportando dos operaciones: unir los conjuntos de dos elementos, y averiguar a qué conjunto pertenece un elemento. Contar componentes conexas es su aplicación más directa, pero la estructura es la base de Kruskal (árbol de recubrimiento mínimo), detección de ciclos en grafos no dirigidos, agrupamiento por equivalencias y problemas de conectividad incremental en general. A diferencia de DFS/BFS, no requiere materializar el grafo: procesa las aristas al vuelo, una por una.

Complejidad: $\mathcal{O}(M\alpha(N))$ para procesar M aristas sobre N nodos, donde α es la inversa de la función de Ackermann (menor que 5 para cualquier entrada imaginable), es decir prácticamente $\mathcal{O}(M)$ amortizado con compresión de caminos.

Fundamento teórico: Cada conjunto se representa como un árbol donde cada nodo apunta a su padre y la RAÍZ (el nodo que es su propio padre) actúa como identificador canónico del conjunto. Dos elementos pertenecen al mismo conjunto exactamente cuando su búsqueda de raíz termina en el mismo nodo, así que el número de componentes conexas es el número de nodos que son su propia raíz. Unir dos conjuntos cuesta una sola asignación: se hace que la raíz de uno apunte a la raíz del otro. La eficiencia depende de mantener los árboles poco profundos, y para eso se usa COMPRESIÓN DE CAMINOS: durante la búsqueda de la raíz, cada nodo visitado se reengancha más arriba en el árbol, de modo que las consultas futuras sobre ese nodo (y sobre todos sus descendientes en el camino) sean más cortas. La variante implementada aquí, *path halving*, hace que cada nodo apunte a su abuelo en lugar de a su padre mientras sube, lo que reduce la profundidad a la mitad en cada pasada con un solo bucle y sin recursión ni segunda pasada. El resultado es que la profundidad amortizada queda acotada por $\alpha(N)$, una función que crece tan lentamente que es constante en la práctica. Nótese que la estructura es naturalmente inmune a aristas repetidas y lazos: si a y b ya comparten raíz, la unión simplemente no hace nada, así que la entrada puede contener CE y EC (como en el ejemplo) o incluso AA sin afectar el conteo.

Ejercicio aplicado

Dado el nombre del nodo mayor del grafo (una letra mayúscula, que implica los nodos desde A hasta esa letra) y una lista de aristas no dirigidas expresadas como pares de letras, contar cuántos subgrafos conexos maximales (componentes conexas) tiene el grafo.

I/O: La entrada separa los casos con LÍNEAS EN BLANCO, lo que normalmente forzaría a leer línea por línea; sin embargo, aquí la estructura por líneas es redundante: el nodo mayor siempre es un token de 1 carácter y cada arista es un token de 2 caracteres. Eso permite usar lectura total + tokenización (patrón 2) y distinguir el inicio de cada caso por la LONGITUD del token, consumiendo aristas mientras midan 2 — el parseo se vuelve inmune a cualquier cantidad de líneas en blanco, espacios extra o retornos de carro. Se opera directamente sobre bytes, restando 64 al código ASCII para mapear 'A'→1 hasta 'Z'→26 (a lo sumo 26 nodos, así que el rendimiento es irrelevante). La salida exige una línea en blanco entre casos consecutivos, lo que se logra con `"\n\n".join(out)`. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta de BFS sobre listas de adyacencia en 120 tandas aleatorias (incluyendo lazos AA y aristas duplicadas), más casos borde de un solo nodo y de casos sin ninguna arista, sin discrepancias.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    t = int(data[idx]); idx += 1
    out = []
    for _ in range(t):
        # el nodo mayor es un token de 1 caracter; las
        # aristas son de 2
        n = data[idx][0] - 64          # 'A'=65 -> 1 nodo
        ... 'Z'=90 -> 26 nodos
        idx += 1
        padre = list(range(n + 1))

        def find(x):
            while padre[x] != x:
                padre[x] = padre[padre[x]] # path
            halving: apunta al abuelo
            x = padre[x]
            return x

        # consume aristas mientras el token siguiente
        # tenga 2 caracteres:
        # así el parseo ignora por completo las líneas en
        # blanco del enunciado
        while idx < len(data) and len(data[idx]) == 2:
            tok = data[idx]; idx += 1
            a = find(tok[0] - 64)
            b = find(tok[1] - 64)
            if a != b:                # si ya comparten
                padre[a] = b          # o lazo) no se
            # cada nodo que es su propia raíz representa una
            # componente
            out.append(str(sum(1 for v in range(1, n + 1) if
                find(v) == v)))
    sys.stdout.write("\n\n".join(out) + "\n")
```

solve()

5 Búsqueda y Optimización

5.1 Búsqueda Binaria sobre la Respuesta + Two Pointers (k-ésima suma de pares)

Preguntas guía:

- ¿La respuesta es un número y existe una condición monótona (si X funciona, todo valor mayor/menor también funciona)?
- ¿Puedo verificar en $\mathcal{O}(n)$ o $\mathcal{O}(n \log n)$ si un valor candidato "cumple" la condición pedida?
- ¿Dentro de esa verificación necesito contar pares/elementos con una condición de suma sobre un arreglo ordenado? (ahí entra Two Pointers)

Utilidad: Cuando no es fácil calcular la respuesta directamente pero sí verificar si un candidato es válido, y esa validez es monótona, se "busca" la respuesta con búsqueda binaria sobre el rango de valores posibles. Two Pointers acelera el conteo en arreglos ordenados de $\mathcal{O}(n^2)$ a $\mathcal{O}(n)$.

Complejidad: Búsqueda binaria: $\mathcal{O}(\log(\text{rango}))$ iteraciones. Cada verificación con Two Pointers: $\mathcal{O}(n)$. Total: $\mathcal{O}(n \log(\text{rango}))$.

Fundamento teórico: La búsqueda binaria sobre la respuesta funciona porque el conjunto de valores "válidos" forma un intervalo continuo (monotonía), reduciendo el espacio de búsqueda a la mitad en cada paso. Two Pointers explota que, en un arreglo ordenado, mover un extremo cambia la condición de suma de forma predecible, evitando recorrer todas las parejas.

Ejercicio aplicado

Dados N precios de categorías de joyas, un cliente que es el K-ésimo en la fila debe comprar dos joyas de categorías distintas sin repetir una combinación ya usada por otro cliente. Hallar el mínimo dinero que garantiza poder comprar: la K-ésima suma más pequeña entre todos los pares de categorías distintas.

I/O: N no está acotado explícitamente pero puede ser grande $\rightarrow$ lectura total + tokenización (patrón 2) para el arreglo de precios.

Código solución

```
import sys

def count_pairs_leq(prices, n, limit):
    # cuenta cuantos pares cumplen arr[i] + arr[j] <= limit
    # usando dos punteros sobre un arreglo YA ORDENADO
    count = 0
    left, right = 0, n - 1
    while left < right:
        if prices[left] + prices[right] <= limit:
            count += right - left  # todos los pares
            con "left" fijo
            left += 1
        else:
            right -= 1
    return count

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
```

```
prices = list(map(int, data[2:2 + n]))
prices.sort()

low = prices[0] + prices[1]  # menor suma
                             posible
high = prices[n - 2] + prices[n - 1]  # mayor suma
                                     posible

while low < high:
    mid = low + (high - low) // 2
    if count_pairs_leq(prices, n, mid) >= k:
        high = mid  # mid cumple:
                    intenta algo menor
    else:
        low = mid + 1  # mid no cumple:
                       sube el limite

print(low)

solve()
```

5.2 Enumeración por Bitmask (bits perdidos de un mensaje)

Preguntas guía:

- ¿Hay un pequeño número ($\leq \sim 20$) de decisiones binarias desconocidas (bits, elecciones sí/no) y hace falta probar combinaciones?
- ¿El espacio total de combinaciones (2^k) es lo bastante chico para probarlas todas dentro del límite de tiempo?
- ¿Cada combinación se puede armar y evaluar rápido a partir de su representación binaria?

Utilidad: Representar y recorrer exhaustivamente todas las configuraciones posibles de un conjunto pequeño de decisiones binarias. Cada máscara de bits codifica una asignación distinta de valores, permitiendo generar candidatos de forma sistemática y verificar cuál satisface la condición del problema.

Complejidad: $\mathcal{O}(2^k \cdot C)$, con k = número de incógnitas binarias y C = costo de evaluar una combinación.

Fundamento teórico: Cualquier asignación de k bits desconocidos corresponde biunívocamente a un entero entre 0 y $2^k - 1$. Recorriendo ese rango y extrayendo el bit j con $(\text{mascara} \gg j) \& 1$, se reconstruye cada combinación posible sin necesidad de recursión ni backtracking explícito. Es la enumeración más directa de un espacio de búsqueda combinatorio pequeño.

Ejercicio aplicado

Un mensaje M' y un divisor N' se recibieron con hasta 16 bits en total marcados como perdidos (*). Reconstruir un mensaje M compatible (M completando los '*' de M') tal que exista un N (completando los '*' de N') donde N divide a M exactamente.

I/O: $|M'| \leq 500$ pero el total de '*' (en M' y N' combinados) es $\leq 16 \rightarrow$ lectura total con readline es más que suficiente; el costo real está en las 2^{16} combinaciones, no en la entrada.

Código solución

```
import sys
```

```

input = sys.stdin.readline

def generate(strings):
    pos = [] # posiciones donde existen bits desconocidos
    for i, c in enumerate(strings):
        if c == '?':
            pos.append(i)
    k = len(pos)

    ans = []
    for mask in range(1 << k): # cada mascara representa una asignacion distinta de los k bits perdidos
        cur = strings[:]
        for j in range(k):
            if (mask >> j) & 1:
                cur[pos[j]] = '1'
            else:
                cur[pos[j]] = '0'
        ans.append(''.join(cur))
    return ans

M = list(input().strip())
N = list(input().strip())

messages = generate(M)
divisors = generate(N)

for m in messages: # probar todas las combinaciones posibles
    value_m = int(m, 2)
    for n in divisors:
        value_n = int(n, 2)

        if value_n == 0:
            continue # la division entre cero no existe
        if value_m % value_n == 0: # la primera combinacion valida es respuesta
            print(m)
            sys.exit()

```

5.3 Búsqueda Binaria sobre la Respuesta: Maximizar el Mínimo (muralla con refuerzo)

Preguntas guía:

- ¿Piden maximizar un mínimo (o minimizar un máximo) tras aplicar UNA modificación elegida de un conjunto de opciones?
- ¿Puedo verificar rápido, para un valor objetivo fijo, si ALGUNA elección lo logra?
- ¿La factibilidad es monótona: si un objetivo H es alcanzable, cualquier objetivo menor también lo es?

Utilidad: Resolver problemas de "maximizar el mínimo" cuando la modificación disponible tiene una forma estructurada (aquí, un refuerzo en forma de escalera), buscando binariamente el mejor valor objetivo y verificando factibilidad con una condición cerrada en $\mathcal{O}(n)$ por candidato.

Complejidad: $\mathcal{O}(n \log(\max x + K))$.

Fundamento teórico: Para un objetivo H fijo, una posición i con $x_i < H$ tiene una *deficiencia* $d_i = H - x_i$.

Si se refuerza en la posición p , la posición i recibe bonificación $K - (p - i)$ solo si $p - K + 1 \leq i \leq p$; para que alcance H , se necesita $p \leq i + K - d_i$. Además, ninguna posición fuera del rango de bonificación puede ser deficiente, lo que obliga a $p \geq R$ (la posición deficiente más a la derecha). Así, el conjunto de posiciones p válidas es la intersección de intervalos $[i, i + K - d_i]$ sobre todas las posiciones deficientes i ; como estos intervalos tienen extremo izquierdo creciente en i , la intersección es no vacía si y solo si $R \leq K + \min_i (i + x_i) - H$ (tomando el mínimo solo entre las posiciones deficientes). Esta condición se evalúa en $\mathcal{O}(n)$, y como la factibilidad de H es monótona (un refuerzo que logra mínimo H automáticamente logra cualquier $H' < H$), se puede buscar binariamente el mayor H factible.

Ejercicio aplicado

Dada una muralla de N segmentos con alturas iniciales, y un refuerzo que agrega K bloques al segmento elegido, $K - 1$ al anterior, y así sucesivamente, hallar la mayor altura mínima posible tras aplicar el refuerzo en un único segmento.

I/O: $N \leq 10^5 \rightarrow$ lectura total con tokenización (patrón 2). Verificado contra fuerza bruta en 300 casos aleatorios pequeños.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    x = list(map(int, data[2:2 + n]))

    def feasible(h):
        # deficientes: posiciones (1-indexadas) por debajo de h
        r = -1 # posicion deficiente mas a la derecha
        best = None # min(i + x[i]) entre las deficientes (1-indexado)
        for i in range(n):
            if x[i] < h:
                pos = i + 1
                r = pos
                val = pos + x[i]
                if best is None or val < best:
                    best = val
        if r == -1:
            return True # nadie por debajo de h: cualquier refuerzo sirve
        # interseccion de rangos validos de p: [pos, pos+K-d] para cada deficiente
        # no vacia <=> r <= K + best - h
        return r <= k + best - h

    lo, hi = 1, max(x) + k
    while lo < hi:
        mid = (lo + hi + 1) // 2 # busca el mayor H factible
        if feasible(mid):
            lo = mid
        else:
            hi = mid - 1
    print(lo)

solve()

```

5.4 Huffman: Fusión Greedy con Heap (teñir telas)

Preguntas guía:

- ¿Debo partir repetidamente un grupo en dos subgrupos (o fusionar repetidamente dos grupos en uno) hasta llegar a elementos individuales, pagando en cada paso el peso total del grupo involucrado?
- ¿El costo total equivale a $\sum w_i \cdot d_i$, donde d_i es la profundidad del elemento i en el árbol binario resultante, y me piden minimizarlo?
- ¿Los elementos pueden reordenarse libremente, de modo que cualquier estructura de árbol binario es alcanzable?

Utilidad: Construye el árbol binario que minimiza la suma ponderada de profundidades $\sum w_i d_i$, equivalente a minimizar el costo acumulado de fusionar (o partir) grupos donde cada operación cuesta el peso total del grupo resultante. Es la solución estándar para compresión por códigos de longitud variable, y aparece disfrazado en muchos problemas de "combinar/dividir pagando la suma" (unir cuerdas, mezclar archivos, partir lotes).

Complejidad: $\mathcal{O}(n \log n)$ por caso de prueba, domina por las $n - 1$ operaciones sobre el heap binario.

Fundamento teórico: El proceso de dividir repetidamente un conjunto en dos partes hasta llegar a singletons define un árbol binario cuyas hojas son los elementos. Cada división paga la suma de los pesos del grupo dividido, es decir, la suma de las hojas de ese subárbol; como un elemento i es pagado una vez por cada nodo interno que lo tiene por debajo, su contribución total es $w_i \cdot d_i$ con d_i su profundidad. Por lo tanto el costo total es $\sum_i w_i d_i$ y el problema es exactamente el de Huffman, que se resuelve construyendo el árbol de abajo hacia arriba: en cada paso se fusionan los DOS pesos menores. La optimalidad descansa en dos lemas. (i) *Lema de intercambio:* en algún árbol óptimo, los dos pesos menores $a \leq b$ pueden colocarse como hermanos en el nivel más profundo — si no lo estuvieran, intercambiarlos con las hojas más profundas no aumenta el costo, pues mover un peso menor hacia mayor profundidad y uno mayor hacia menor profundidad tiene delta $(w_{\text{mayor}} - w_{\text{menor}})(d_{\text{menor}} - d_{\text{mayor}}) \leq 0$. (ii) *Subestructura óptima:* reemplazar esas dos hojas hermanas por un único nodo de peso $a + b$ produce una instancia con un elemento menos cuyo costo óptimo difiere del original exactamente en la constante $a + b$, de modo que resolver óptimamente la instancia reducida resuelve óptimamente la original. La inducción sobre estos dos lemas da la corrección del greedy. Nótese que el enunciado permite disponer los elementos en cualquier orden en la banda, lo cual garantiza que CUALQUIER estructura de árbol binario (no solo particiones contiguas de un orden fijo) es realizable, así que no hay restricción adicional sobre la forma del árbol.

Ejercicio aplicado

Dadas N telas con sus tamaños, todas inicialmente del mismo color, y un proceso que en cada paso toma todas las telas de un color y las reparte en dos colores nuevos pagando la suma de sus tamaños, calcular la tinta mínima total necesaria para que todas las telas terminen con colores distintos.

I/O: Hasta $T \leq 100$ casos con $\sum N_i \leq 10^5$ y tamaños hasta $10^5 \Rightarrow$ lectura total + tokenización con índice `idx` (patrón 2), ya que son muchos números sueltos repartidos en líneas de longitud variable. El caso $N = 1$ se atiende por separado (costo 0: no hace falta ninguna división). La respuesta puede superar los 32 bits, pero los enteros nativos de Python no requieren cuidado especial. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta que explora TODAS las particiones recursivas posibles (cualquier subconjunto, no solo contiguos) en 200 casos aleatorios con $N \leq 8$, sin discrepancias. Prueba de estrés con $\sum N = 10^5$: ~ 0.14 s con un solo caso de $N = 10^5$, ~ 0.06 s repartido en 100 casos, y ~ 0.07 s con todos los tamaños iguales (máximo de empates en el heap).

Código solución

```
import sys
import heapq

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    t = int(data[idx]); idx += 1
    out = []
    for _ in range(t):
        n = int(data[idx]); idx += 1
        sizes = data[idx:idx + n]
        idx += n
        if n == 1:
            out.append("0") # una sola tela: ya
                           # tiene color unico, no se gasta tinta
            continue
        heap = [int(x) for x in sizes]
        heapq.heapify(heap) # heapify es O(n), mas
                           # rapido que n heappush
        total = 0
        while len(heap) > 1:
            a = heapq.heappop(heap)
            b = heapq.heappop(heap)
            s = a + b
            total += s # cada fusion paga el
                     # peso del grupo resultante
            heapq.heappush(heap, s)
            out.append(str(total))
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

5.5 DP de Particionamiento sobre Residuos (compra con redondeo)

Preguntas guía:

- ¿Debo partir una secuencia en grupos contiguos minimizando una suma de costos por grupo, con un límite en la cantidad de cortes?
- ¿El costo de cada grupo es (suma del grupo) más un ajuste que depende SOLO de la suma módulo algo pequeño (redondeo, paridad, comisión por tramos)?

- ¿El DP directo sobre posiciones sería $\mathcal{O}(n^2d)$ y los límites lo hacen inviable en Python?
- ¿Puedo reemplazar el estado "posición del último corte" por "residuo del prefijo en el último corte", colapsando n estados en unos pocos?

Utilidad: Reduce drásticamente el espacio de estados de un DP de particionamiento cuando el costo de cada bloque se descompone en una parte TELESCÓPICA (que se cancela globalmente y no depende de dónde se corte) más una parte que solo depende de un residuo módulo un número pequeño. El estado deja de ser "dónde fue el último corte" (n posibilidades) y pasa a ser "qué residuo tenía el prefijo en el último corte" (10 posibilidades), convirtiendo un $\mathcal{O}(n^2d)$ en $\mathcal{O}(ndM)$ con M el módulo.

Complejidad: $\mathcal{O}(n \cdot d \cdot M)$ con $M = 10$, es decir $\mathcal{O}(nd)$ para efectos prácticos, frente a $\mathcal{O}(n^2d)$ del DP directo.

Fundamento teórico: Sea P_i la suma de prefijos y $\text{red}(x)$ el redondeo de x al múltiplo de 10 más cercano (con 5 hacia arriba). Escribiendo $x = 10q + m$ con $m = x \bmod 10$, se tiene $\text{red}(x) = x + \delta(m)$, donde $\delta(m) = -m$ si $m < 5$ y $\delta(m) = 10 - m$ si $m \geq 5$; es decir, el redondeo es la identidad MÁS un ajuste que depende únicamente del residuo. Si los cortes son $0 = c_0 < c_1 < \dots < c_k = n$ y $\Delta_t = P_{c_t} - P_{c_{t-1}}$ es la suma del grupo t , el costo total es

$$\begin{aligned} \sum_{t=1}^k \text{red}(\Delta_t) &= \sum_{t=1}^k \Delta_t + \sum_{t=1}^k \delta(\Delta_t \bmod 10) \\ &= P_n + \sum_{t=1}^k \delta((r_{c_t} - r_{c_{t-1}}) \bmod 10), \end{aligned}$$

donde la primera suma TELESCOPIA a la constante P_n (independiente de los cortes) y $r_i = P_i \bmod 10$. La consecuencia clave es que el término a minimizar depende de las posiciones de corte ÚNICAMENTE a través de los residuos de sus prefijos, porque $\Delta_t \bmod 10 = (r_{c_t} - r_{c_{t-1}}) \bmod 10$. Por eso el estado del DP puede ser $(j, r) =$ "ya se formaron j grupos y el último corte estaba en una posición con residuo r ", guardando el ajuste acumulado mínimo; la posición exacta del corte es irrelevante más allá de su residuo y de que sea anterior a la posición actual. Un detalle de implementación importante: todos los candidatos de la posición i deben calcularse a partir del estado ANTERIOR a actualizarla, para no encadenar un corte con otro de la misma posición (grupo vacío); dicho sea de paso, permitir grupos vacíos sería inofensivo en cuanto al valor óptimo, pues $\delta(0) = 0$ y equivale a gastar un divisor sin ganancia, lo cual el propio enunciado admite al decir "hasta d divisores".

Ejercicio aplicado

Dados n precios en el orden en que están sobre la banda y hasta d divisores, partir la compra en a lo sumo $d + 1$ grupos contiguos y pagar cada grupo por separado, donde el total de cada grupo se redondea al múltiplo de 10 más cercano. Minimizar el monto total pagado.

I/O: $n \leq 2000$, $d \leq 20$, precios hasta 10^4 , en dos líneas de muchos números sueltos $\Rightarrow$ lectura total + tokenización

(patrón 2). Verificado contra los dos casos de ejemplo y contrastado con el DP directo $\mathcal{O}(n^2d)$ en 400 casos aleatorios pequeños, sin discrepancias, más casos borde (un solo item, un item que redondea hacia arriba, más divisores que items, todos los precios múltiplos de 10). Sobre el rendimiento conviene ser explícito: el DP directo, implementado en Python puro, tarda ~ 5.60 s en el peor caso ($n = 2000$, $d = 20$), justo POR ENCIMA del límite de 5 segundos; el DP sobre residuos resuelve el mismo caso en ~ 0.04 s (unas 140 veces más rápido) y ambos coinciden en la respuesta. Es decir, aquí la reducción de estados no es una optimización cosmética sino la diferencia entre TLE y AC.

Código solución

```
import sys

data = sys.stdin.buffer.read().split()
n = int(data[0]); divs = int(data[1])
acumulado = 0
pref = [0]*(n+1)
for i in range(n):
    acumulado += int(data[2+i])
    pref[i+1] = acumulado
total = acumulado # parte telescópica:
                    constante

ajuste = [0,-1,-2,-3,-4,5,4,3,2,1]

INF = float('inf')
max_grupos = divs + 1 # a lo
                    sumo d+1 grupos
# ajuste_min[j][r] = ajuste mínimo acumulado tras formar
# j grupos,
# con el último corte en una posición cuyo prefijo tiene
# residuo r
ajuste_min = [[INF]*10 for _ in range(max_grupos+1)]
ajuste_min[0][pref[0] % 10] = 0

mejor = INF
for i in range(1, n+1):
    residuo_curr = pref[i] % 10
    # se calculan TODOS los candidatos con la g anterior,
    # para no
    # encadenar un corte con otro de esta misma posición
    # (grupo vacío)
    cand = [INF]*(max_grupos+1)
    for j in range(1, max_grupos+1):
        fila = ajuste_min[j-1]
        minimum = INF
        for r in range(10):
            v = fila[r]
            if v < INF:
                v += ajuste[(residuo_curr - r) % 10]
                if v < minimum: minimum = v
            cand[j] = minimum
    for j in range(1, max_grupos+1):
        if cand[j] < ajuste_min[j][residuo_curr]:
            ajuste_min[j][residuo_curr] = cand[j]
    if i == n:
        mejor = min(cand[1:]) # el último grupo debe
                                cerrar en n
print(total + mejor)
```

6 Matemáticas y Simulación

6.1 Reducción Binaria tipo Josefo (fila con detector alternado)

Preguntas guía:

- ¿El problema describe un proceso repetitivo/alternado (cada segundo elemento se "marca") sobre n elementos, con n muy grande ($\geq 10^9$)?
- ¿Simular paso a paso sería $\mathcal{O}(n)$, demasiado lento, pero el patrón se repite de forma autosimilar al dividir por la mitad?
- ¿Puedo replantear el problema de tamaño n como un subproblema de tamaño $\sim n/2$ aplicando la misma regla?

Utilidad: Resolver en $\mathcal{O}(\log n)$ problemas de simulación con patrones alternados sobre secuencias enormes, sin simular cada paso. Es la misma idea del clásico Problema de Josefo: en vez de eliminar uno por uno, se "salta" de una ronda completa a la mitad del problema restante.

Complejidad: $\mathcal{O}(\log n)$ por consulta (cada ronda aprox. divide n a la mitad).

Fundamento teórico: En cada "ronda" del proceso, la mitad de los elementos son descartados/activados según la paridad de su posición. Reformulando la posición y el tamaño del problema tras cada ronda ($n' = n - \text{procesados}$, $i' = \text{nueva posición relativa}$) se llega a una recurrencia que converge en $\mathcal{O}(\log n)$ pasos en vez de $\mathcal{O}(n)$.

Ejercicio aplicado

Hay N personas en fila y un detector de metales defectuoso que se activa de forma alternada (la 1ª persona no lo activa, la 2ª sí, la 3ª no, ...; quien lo activa va al final de la fila). Dado que el coach está en la posición i , determinar en qué turno (entre quienes no activan el detector) pasará.

I/O: $T \leq 1000$ casos, cada uno con solo dos enteros $\rightarrow$ readline por línea (patrón 1) es suficiente y más simple.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def safe_position(n, i):
    # turno "seguro" (que no activa el patron) en que pasara
    # la posicion i
    res = 0
    while i % 2 == 0:          # mientras i cae en la
        mitad "activada"      # cuenta los turnos seguros
        res += i // 2
        de esta ronda
        n -= i // 2            # reduce el problema a la
        mitad restante
        i = n                  # la posicion i se
        convierte en el nuevo n
    res += (i + 1) // 2        # suma los turnos seguros
    de la ronda final
    return res

def solve():
    t = int(input())
    out = []
    for _ in range(t):
        n, i = map(int, input().split())
        out.append(str(safe_position(n, i)))
```

```
sys.stdout.write("\n".join(out))
```

```
solve()
```

6.2 Fuerza Bruta / Simulación Directa (paradoja del cumpleaños)

Preguntas guía:

- ¿El tamaño de la entrada es pequeño ($N \leq 10^3$ o similar) y no hay una fórmula cerrada evidente?
- ¿Puedo calcular la respuesta iterando directamente sobre el espacio del problema sin que se dispare el tiempo?
- ¿El problema es más de matemáticas/probabilidad que de estructuras de datos?

Utilidad: Reconocer cuándo NO hace falta un algoritmo sofisticado: con restricciones pequeñas, iterar directamente (simulando la definición del problema) es la solución más simple, rápida de programar y menos propensa a errores. Identificar esto ahorra tiempo valioso en el contrarreloj.

Complejidad: Depende del problema; en este caso $\mathcal{O}(n)$ con aritmética de punto flotante.

Fundamento teórico: Se calcula directamente la probabilidad de que NO haya coincidencia de cumpleaños entre k personas, multiplicando factores $(1 - (k-1)/n)$ sucesivamente (extensión del problema del cumpleaños), deteniéndose apenas esa probabilidad cae por debajo de 0.5 (la probabilidad de coincidencia supera el 50%).

Ejercicio aplicado

Dado N (número de días del año), hallar el mínimo número de personas necesarias en una sala para que la probabilidad de que dos compartan cumpleaños supere el 50%.

I/O: Entrada de un solo entero $\rightarrow$ readline (patrón 1), no requiere nada especial.

Código solución

```
import sys
input = sys.stdin.buffer.readline

n = int(input())
prob_no_coincidencia = 1.0
for k in range(1, n + 2):
    prob_no_coincidencia *= (n - (k - 1)) / n
    if prob_no_coincidencia < 0.5:
        print(k)
        break
```

6.3 Simulación Lineal con Máximo de Pre-fijo (paredes que se acercan)

Preguntas guía:

- ¿La respuesta depende de un solo recorrido lineal del arreglo, calculando un máximo/mínimo/acumulado por posición?
- ¿No hay dependencias entre posiciones más allá de una comparación directa fila por fila?

- ¿El problema, aunque tenga una narrativa elaborada, se reduce matemáticamente a una fórmula simple tras identificar la variable clave?

Utilidad: Resolver directamente problemas donde, tras modelar bien la situación (aquí: el "cuello de botella" es la fila con menor espacio libre entre dos paredes), un solo recorrido $\mathcal{O}(n)$ calculando un máximo/mínimo basta. Es clave en competencia reconocer cuándo un problema de apariencia compleja se reduce a esto.

Complejidad: $\mathcal{O}(h)$ tiempo, $\mathcal{O}(h)$ espacio (o $\mathcal{O}(1)$ extra si se procesa en streaming).

Fundamento teórico: Las dos paredes se mueven a 1 m/s cada una. En la fila i , el espacio libre es $w - (l[i] + r[i])$ y se cierra a razón de 2 m/s (ambas paredes se acercan). El tiempo en que la fila i se cierra es $(w - l[i] - r[i]) / 2$. Las paredes chocan en la fila que se cierra primero: la de menor espacio libre inicial, equivalente al máximo de $(l[i] + r[i])$.

Ejercicio aplicado

Una sala de H filas y ancho W tiene una pared este y una pared oeste que se acercan a 1 m/s cada una desde que alguien entra. Cada fila i tiene un ancho ya ocupado por cada pared. Determinar en cuánto tiempo chocan las paredes.

I/O: $H \leq 10^5$, dos arreglos de tamaño $H \rightarrow$ lectura total + tokenización (patrón 2).

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    h = int(data[0]); w = int(data[1])
    left = list(map(int, data[2:2 + h]))
    right = list(map(int, data[2 + h:2 + 2 * h]))
    max_sum = max(left[i] + right[i] for i in range(h))
    ans = (w - max_sum) / 2

if ans.is_integer():           # verifica parseo de float a
    int                         int
    print(int(ans))
else:
    print(ans)
solve()
```

6.4 Simulación de Eventos con Colas (escalera doble)

Preguntas guía:

- ¿Hay un recurso compartido con estado (dirección, disponibilidad) que cambia según el orden de llegada de eventos con timestamps?
- ¿Los eventos ya vienen ordenados (o se pueden ordenar) y conviene procesarlos con colas separadas por tipo?
- ¿El estado siguiente depende de comparar el próximo evento contra un tiempo límite (fin de uso actual del recurso)?

Utilidad: Simular un sistema con estado (aquí: una dirección de movimiento y un instante de fin de uso) que evoluciona a medida que llegan eventos de distintos tipos, usando una cola por tipo de evento en vez de simular minuto a minuto.

Complejidad: $\mathcal{O}(n)$ - cada persona se encola y desencola una sola vez.

Fundamento teórico: Se mantienen dos colas (una por dirección deseada) y un estado (dirección activa, instante en que el recurso queda libre). En cada paso se mira el evento más próximo entre ambas colas: si coincide con la dirección activa, extiende el fin de uso; si es la dirección contraria, esa persona debe esperar (se cuenta pero no se procesa aún); si ya no hay eventos antes de que el recurso quede libre, se decide si invertir la dirección (si había gente esperando) o terminar ese ciclo de uso.

Ejercicio aplicado

N personas llegan a una escalera doble (10 segundos por cruce) en tiempos distintos, cada una queriendo ir en una de dos direcciones. Determinar el instante en que la última persona termina de usarla.

I/O: $n \leq 10^4 \rightarrow$ lectura total + tokenización (patrón 2); se usan dos deque (collections.deque) en vez de queue, ya que en Python deque tiene $\mathcal{O}(1)$ real para popleft().

Código solución

```
import sys
from collections import deque

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); idx += 1
    left = deque()      # personas que quieren ir en
                        # direccion 0
    right = deque()     # personas que quieren ir en
                        # direccion 1
    for _ in range(n):
        t = int(data[idx]); d = int(data[idx + 1]); idx += 2
        (left if d == 0 else right).append(t)

    end = 0             # instante en que el recurso queda
                        # libre
    ans = 0
    waiting = 0         # cuantos esperan en direccion
                        # contraria a la activa
    direction = -1      # -1 = detenida

    while left or right or direction != -1:
        # mira el proximo evento disponible entre ambas
        # colas
        next_time, who = None, -1
        if left and (next_time is None or left[0] <
next_time):
            next_time, who = left[0], 0
        if right and (next_time is None or right[0] <
next_time):
            next_time, who = right[0], 1

        if direction == -1:
            # arranca detenida: la toma quien llegue
            # primero
            direction = who
            end = next_time + 10
            (left if who == 0 else right).popleft()
```

```

elif who != -1 and next_time < end:
    if who == direction:
        end = max(end, next_time + 10) # se
        sube mientras sigue en marcha
    else:
        waiting += 1 #
        direccion contraria: debe esperar
        (left if who == 0 else right).popleft()
    else:
        # no hay mas eventos antes de que el recurso
        quede libre
        if waiting > 0:
            direction = 1 - direction # invierte
            sentido para atender a los que esperan
            end += 10
            waiting = 0
        else:
            ans = end
            direction = -1 # se detiene

print(ans)
solve()

```

6.5 Simulación XOR con Detección de Ciclos (interruptores y bombillas)

Preguntas guía:

- ¿El estado del sistema se puede representar como un vector de bits, y cada operación disponible es un XOR con una máscara fija?
- ¿Las operaciones se aplican en un orden CÍCLICO fijo y predecible (no libre elección)?
- ¿Necesito detectar si el proceso entra en un ciclo infinito sin nunca alcanzar el estado objetivo?

Utilidad: Simular un proceso determinista donde el estado (aquí, qué bombillas están encendidas) cambia por XOR con máscaras fijas aplicadas en orden cíclico, deteniéndose al llegar al estado objetivo o detectando que nunca se alcanzará. Es aplicable a cualquier proceso de estado finito con transiciones deterministas.

Complejidad: $\mathcal{O}(N)$ en la práctica (ver fundamento teórico), a pesar de que el espacio de estados aparenta ser exponencial en M .

Fundamento teórico: Presionar los N interruptores en orden fijo $1..N$ corresponde a aplicar sucesivamente XOR con N máscaras fijas. Después de un CICLO COMPLETO (los N interruptores), el efecto neto sobre el estado es XOR con una constante fija $C = \text{XOR de todas las máscaras de los interruptores}$, sin importar el estado en que se esté. Como $v \oplus C \oplus C = v$, el estado al INICIO de cada ciclo solo puede tomar 2 valores posibles (el original, o el original $\oplus C$), alternando entre ambos. Esto acota el número de estados distintos alcanzables (y por lo tanto el número de pasos antes de repetir un estado o llegar a 0) a como mucho, $2N$: por eso la detección de ciclos con un conjunto de estados visitados nunca necesita más de un puñado de iteraciones, sin importar cuán grande sea M .

Ejercicio aplicado

Un conserje presiona N interruptores en orden cíclico $1, 2, \dots, N, 1, 2, \dots$. Cada interruptor alterna un subconjunto fijo de M bombillas. Dado el estado inicial de las bombillas encendidas, determinar cuántas veces hay que

presionar interruptores hasta que TODAS las bombillas queden apagadas simultáneamente, o -1 si nunca ocurre.

I/O: $N, M \leq 1000 \rightarrow$ readline por línea (patrón 1) es más que suficiente, dado que el propio algoritmo termina en a lo más $\sim 2N$ iteraciones (verificado: $< 0.03s$ incluso en el peor caso $N=M=1000$).

Código solución

```

import sys
input = sys.stdin.buffer.readline

n, m = map(int, input().split())
ldata = list(map(int, input().split())) # ldata[0]=
        cantidad, resto=bombillas encendidas

switch = []
for i in range(n):
    sdata = list(map(int, input().split()))
    tmp = 0
    for j in range(1, sdata[0] + 1):
        aux = 1 << (m - sdata[j]) # bit que
        representa esa bombilla
        tmp += aux
    switch.append(tmp) # mascara XOR
    de este interruptor

bulbs = 0
for j in range(1, ldata[0] + 1):
    aux = 1 << (m - ldata[j])
    bulbs += aux # estado
    inicial como bitmask

idx = 0
res = 0
seen = set() # detecta si (
    estado, interruptor actual) se repite
while True:
    if bulbs == 0:
        break # todas las
        bombillas apagadas: listo
    if (bulbs, idx) in seen:
        res = -1 # ya visitamos
        este estado exacto: nunca terminara
        break
    seen.add((bulbs, idx))
    info = switch[idx]

    bulbs ^= info # aplica el
    interruptor actual

    if idx + 1 == n:
        idx = 0 # vuelve al
        primer interruptor (orden ciclico)
    else:
        idx += 1
    res += 1

sys.stdout.write(str(res))

```

7 Construcción y Combinatoria

7.1 Constructivo por Invariante de Paridad (permutación con signos)

Preguntas guía:

- ¿El problema pide construir explícitamente una configuración (o probar que es imposible), en vez de optimizar o contar?
- ¿Puedo idear una construcción simple (simetrías, intercambios, patrones repetidos) en vez de buscar entre todas las opciones?
- ¿La imposibilidad de otros valores se puede demostrar con un invariante (paridad, módulo, suma fija) que ninguna elección puede romper?

Utilidad: Resolver problemas donde no hace falta ningún algoritmo de búsqueda: basta con proponer una construcción explícita y justificar, con un argumento de invariante, que el resultado contrario (o cualquier otro valor puntual) nunca puede ocurrir sin importar qué elecciones se hagan.

Complejidad: $\mathcal{O}(n)$ por caso - construir y emitir la respuesta.

Fundamento teórico: Se particiona $\{1, \dots, n\}$ en pares consecutivos $(2k-1, 2k)$ y se intercambian entre sí. Cada par contribuye al contador un valor relacionado con $V_k = (2k-1)(2k)$, que siempre es par; y ese aporte entra a la suma final como $-V_k$, 0 o V_k (las tres opciones son pares). Como la suma de términos pares es par, el resultado nunca puede ser impar - en particular, nunca puede ser 1.

Ejercicio aplicado

(Codeforces 2246A - farmpiggy and Subset Sum) Dado un entero par n , construir una permutación p de longitud n tal que, sin importar qué signo (sumar, restar o ignorar) se le asigne a cada término $i \cdot p_i$ al acumular un contador, el resultado final nunca pueda ser exactamente 1.

I/O: $t \leq 25$, $n \leq 50$ (par) $\rightarrow$ entrada pequeña, readline por línea (patrón 1) es suficiente.

Código solución

```
import sys
input = sys.stdin.buffer.readline

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    p = []
    for i in range(1, n + 1):
        # empareja (1,2), (3,4), ..., (n-1,n) e
        # intercambia cada pareja:
        # el aporte de cada pareja al contador siempre es
        # par (ver teoria)
        if i % 2 == 1:
            p.append(i + 1)      # posicion impar ->
            # apunta a la siguiente
        else:
            p.append(i - 1)      # posicion par -> apunta
            # a la anterior
    out.append(' '.join(map(str, p)))
sys.stdout.write('\n'.join(out) + '\n')
```

7.2 Constructivo por Construcción Explícita (suma divisible por cada elemento)

Preguntas guía:

- ¿Piden construir explícitamente n valores con una relación de divisibilidad o proporción exacta entre ellos y su suma?
- ¿La condición se puede reescribir como una ecuación entre fracciones que deben sumar exactamente 1?
- ¿Existe una descomposición numérica conocida (como una fracción egipcia) que se pueda extender o escalar para cualquier n ?

Utilidad: Construir directamente arreglos o valores que cumplan una condición de divisibilidad o proporción exacta, usando descomposiciones numéricas conocidas en vez de búsquedas o heurísticas. Es frecuente en problemas "constructivos" donde la solución es una fórmula cerrada, no un algoritmo de recorrido.

Complejidad: $\mathcal{O}(n)$ por caso.

Fundamento teórico: Si cada $a_i = S / k_i$ (S = suma total), la condición " S divisible por cada a_i " equivale a que los k_i sean enteros positivos distintos con $\sum(1/k_i) = 1$. Partiendo de la identidad $1 = 1/2 + 1/3 + 1/6$ y extendiendo el término $1/6$ con una cadena de potencias de 2, se obtiene una familia de k_i distintos para cualquier $n \geq 3$: la construcción cerrada $\{1, 2, 3, 6, 12, 24, \dots, 3 \cdot 2^{n-3}\}$. Los casos $n=1$ y $n=2$ se resuelven aparte (trivial e imposible, respectivamente).

Ejercicio aplicado

(Codeforces 2246B - ezraft and Array) Dado n , construir un arreglo de n enteros positivos distintos tal que la suma total sea divisible por cada uno de sus elementos, o determinar que no es posible.

I/O: $t \leq 50$, $n \leq 50 \rightarrow$ entrada pequeña, readline por línea (patrón 1).

Código solución

```
import sys
input = sys.stdin.buffer.readline

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    if n == 1:
        out.append('1')      # caso base: un
                              # unico elemento
    elif n == 2:
        out.append('-1')      # a1/a2 y a2/a1
                              # con ambos positivos
                              # obliga a1 ==
                              # a2 (no son distintos)
    else:
        # construccion cerrada a partir de 1 = 1/2 + 1/3
        # + 1/6:
        # {1, 2} mas una cadena de potencias de 2
        # escaladas por 3,
        # que nunca "chocan" con los divisores de 1 o 2
        vals = [1, 2] + [3 * (1 << k) for k in range(n - 2)]
        out.append(' '.join(map(str, vals)))
sys.stdout.write('\n'.join(out) + '\n')
```

7.3 Conteo por Simetría de Binomios (subsecuencias con suma alternada 0)

Preguntas guía:

- ¿Piden contar subconjuntos o subsecuencias que cumplan una condición de suma o balance?
- ¿Hay valores repetidos en la entrada, de forma que agruparlos por valor simplifique el conteo?
- ¿Se puede usar $C(c, \text{par}) = C(c, \text{impar}) = 2^{c-1}$ para separar el conteo por grupo y combinar con una potencia de 2?

Utilidad: Contar eficientemente subconjuntos o subsecuencias que satisfacen una condición combinatoria, evitando enumerar los 2^n subconjuntos, aprovechando la simetría de los coeficientes binomiales para reducir el problema a una subsecuencia mucho más pequeña: solo los valores distintos.

Complejidad: $\mathcal{O}(n)$ por caso (la entrada ya viene no-decreciente).

Fundamento teórico: Dentro de un grupo de c copias iguales, elegir una cantidad par de ellas no cambia la suma alternada acumulada (aporta 0) y elegir una cantidad impar sí la cambia (aporta $\pm \text{valor}$). Como $C(c, \text{par}) = C(c, \text{impar}) = 2^{c-1}$, cada grupo aporta el mismo peso sin importar el modo elegido, así que el problema se reduce a contar subconjuntos de suma alternada 0 sobre los valores DISTINTOS, multiplicando el resultado por 2^{n-m} al final (m = cantidad de valores distintos). En esa secuencia reducida, agrupando términos de dos en dos se demuestra que la única forma de obtener alternada 0 con más de un elemento es usando exactamente 3 valores: -1 y un par de enteros positivos consecutivos.

Ejercicio aplicado

(Codeforces 2246C - Omar and Alternating Sums) Dado un arreglo no decreciente de n valores (cada uno -1 o un entero positivo), contar cuántas subsecuencias tienen suma alternada (primer término positivo, segundo negativo, tercero positivo, ...) igual a 0, módulo $10^9 + 7$.

I/O: $t \leq 10^4$, $\sum n \leq 2 \cdot 10^5 \rightarrow$ volumen moderado; se usa readline por línea (patrón 1), adecuado porque el costo dominante está en el procesamiento $\mathcal{O}(n)$, no en la lectura.

Código solución

```
import sys
input = sys.stdin.buffer.readline

MOD = 10**9 + 7

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))

    # el arreglo ya viene no-decreciente: colapsar a
    # valores distintos.
    # cada grupo de c repetidos aporta un factor 2^(c-1)
    # independiente
    # del resto (C(c, par) == C(c, impar) == 2^(c-1))
    distinct = []
    for x in a:
```

```
        if not distinct or distinct[-1] != x:
            distinct.append(x)

    m = len(distinct)
    has_neg1 = 1 if distinct[0] == -1 else 0 # -1 es el
    # unico valor <= 0 posible

    pos = [v for v in distinct if v > 0]
    count_consec = 0
    for i in range(len(pos) - 1):
        if pos[i + 1] - pos[i] == 1: # pares
            # de enteros positivos consecutivos
            count_consec += 1

    # en la secuencia reducida, la suma alternada solo
    # puede dar 0 con:
    # el subconjunto vacio, o exactamente {-1, v, v+1}
    # para cada pareja
    # consecutiva (si hay un -1 disponible)
    reduced = (1 + (count_consec if has_neg1 else 0)) % MOD

    ans = reduced * pow(2, n - m, MOD) % MOD #
    # reincorpora los repetidos
    out.append(str(ans))

sys.stdout.write('\n'.join(out) + '\n')
```

7.4 Greedy por Alternancia con Límite de Racha (dónde dormir cada noche)

Preguntas guía:

- ¿Hay que construir explícitamente una secuencia que respete un límite de repeticiones CONSECUTIVAS de cada elemento (sin límite en el total de usos)?
- ¿Un elemento puede reutilizarse libremente después de "descansar" al menos un turno?
- ¿Basta con dos "recursos" (el mejor y el segundo mejor) para romper cualquier racha, sin necesidad de planear la secuencia completa de antemano?

Utilidad: Construir una secuencia válida de longitud D usando N opciones con límites de racha consecutiva, sin necesidad de una búsqueda ni un algoritmo sofisticado: alternar entre las DOS opciones con mayor límite es siempre suficiente (si el problema es factible), porque cualquier racha se puede "romper" con un solo turno de la segunda opción antes de retomar la primera.

Complejidad: $\mathcal{O}(D)$.

Fundamento teórico: El problema es imposible si ningún lugar admite ni una sola noche (todos los límites son 0), o si solo UN lugar tiene límite positivo y ese límite es menor que D (no hay forma de "descansar" de él). En cualquier otro caso (al menos dos lugares con límite > 0), siempre es posible: se usa el lugar de mayor límite hasta agotar su racha, se intercala un solo día en el segundo mejor lugar (lo cual reinicia la racha del primero), y se repite este patrón hasta completar los D días. Este patrón simple es suficiente porque no hay ningún límite en el número TOTAL de veces que se puede volver a un lugar, solo en la racha consecutiva.

Ejercicio aplicado

Kai necesita dormir D días seguidos, alternando entre N lugares, donde el lugar i tolera a lo más d_i noches

CONSECUTIVAS antes de expulsarlo (pero puede volver después). Construir un plan válido día a día, o determinar que es imposible.

I/O: $N, D \leq 10^5 \rightarrow$ lectura total con tokenización (patrón 2). Verificado contra 200 casos aleatorios comparando factibilidad y validando la secuencia construida.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); d = int(data[1])
    arr = list(map(int, data[2:2 + n]))

    order = sorted(range(n), key=lambda i: -arr[i])
    best_idx = order[0]
    best_val = arr[best_idx]

    if best_val == 0:
        # ningun lugar es utilizable
        print(-1)
        return

    second_val = arr[order[1]] if n > 1 else 0

    if second_val == 0:
        # solo hay UN lugar utilizable: hay que quedarse
        # aqui todos los D dias seguidos
        if best_val >= d:
            print(' '.join([str(best_idx + 1)] * d))
        else:
            print(-1)
        return

    second_idx = order[1]
    out = []
    remaining = d
    while remaining > 0:
        if remaining == 1:
            out.append(second_idx + 1)
            # un dia de "descanso" en el segundo mejor lugar
            break

        out.append(second_idx + 1)
        # un dia de "descanso" en el segundo mejor lugar
        remaining -= 1

        take = min(best_val, remaining)
        out.extend([best_idx + 1] * take)
        remaining -= take
    sys.stdout.write(' '.join(map(str, out)) + '\n')

solve()
```

7.5 Stars and Bars con Factoriales Precalculados (días de espera entre oasis)

Preguntas guía:

- ¿Debo distribuir una cantidad fija de elementos idénticos entre varios lugares?
- ¿Cada lugar puede recibir cero o más elementos?
- ¿El orden dentro de cada grupo no importa, únicamente cuántos elementos recibe cada uno?

Utilidad: Contar el número de formas de distribuir una cantidad fija de elementos indistinguibles entre varios grupos independientes. Es una de las aplicaciones más comunes de la técnica de *Stars and Bars* (estrellas y barras),

utilizada frecuentemente en problemas de combinatoria, programación competitiva y teoría de conteo.

Complejidad: Preprocesamiento de factoriales $\mathcal{O}(MAX)$; cada consulta $\mathcal{O}(1)$; espacio $\mathcal{O}(MAX)$.

Fundamento teórico: Si el viaje mínimo hasta el oasis destino requiere O días y debe durar exactamente T días, existen $extra = T - O$ días de espera que pueden distribuirse libremente antes de salir o después de cualquiera de los D oasis recorridos, es decir, entre $D + 1$ posiciones distintas. Como los días de espera son indistinguibles y cada posición puede recibir cualquier cantidad de ellos, el problema se reduce directamente al principio de *Stars and Bars*, obteniendo un total de $\binom{extra+D}{D}$ planes posibles.

Ejercicio aplicado

Se conocen las posiciones de todos los oasis de una ruta. Para cada consulta se recibe un oasis destino y el número exacto de días que debe durar el viaje. Calcular cuántos planes distintos existen para distribuir los días de espera manteniendo siempre el mismo recorrido.

I/O: $5 \leq N \leq 20$, $1 \leq M \leq 10$. Las distancias a los oasis no superan los 60 días, por lo que todos los factoriales necesarios pueden precomputarse una única vez antes de responder las consultas; cada consulta se responde en tiempo constante.

Código solución

```
import sys
input = sys.stdin.buffer.readline
cant, queries = map(int, input().split()) # numero de oasis y consultas
oasis = list(map(int, input().split())) # distancia minima hasta cada oasis

MAX = 80
factorial = [1] * (MAX + 1) # factoriales para responder combinaciones en O(1)

for i in range(1, MAX + 1):
    factorial[i] = factorial[i - 1] * i

def combination(n, k):
    return factorial[n] // (factorial[k] * factorial[n - k])

for _ in range(queries):
    destiny, time = map(int, input().split())
    extraTime = time - oasis[destiny - 1] # dias extra que deben distribuirse como esperas
    res = combination(extraTime + destiny, destiny) # Stars and Bars
    sys.stdout.write(str(res) + "\n")
```

8 Algoritmos de Cadenas (Strings)

8.1 Balance de Paréntesis por Bloques de Nivel Superior (parejas incompatibles)

Preguntas guía:

- ¿Trabajo con una secuencia balanceada de paréntesis y necesito razonar sobre qué emparejamientos específicos son válidos?
- ¿La pregunta involucra qué pasa con el RESTO de la secuencia al fijar/quitar un par específico?
- ¿Puedo identificar los "bloques de nivel superior" (tramos donde el balance nunca vuelve a cero hasta el final del bloque)?

Utilidad: Analizar, para una secuencia balanceada de paréntesis, la validez de emparejamientos NO necesariamente anidados de la forma estándar, descomponiendo la secuencia en bloques de nivel superior (tramos maximales donde el balance se mantiene estrictamente positivo hasta el cierre). La clave es que un '(' y un ')' se pueden emparejar sin romper la validez del resto si y solo si están en el MISMO bloque.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: Sea $\text{bal}(p)$ el balance (número de '(' menos número de ')') tras procesar los primeros p caracteres. Un bloque de nivel superior termina exactamente donde $\text{bal}(p) = 0$. Al emparejar una posición i '(' con una posición j ')', $i < j$, el balance de las posiciones ENTRE i y $j - 1$ baja en 1 respecto al original (porque ya se "usó" el '(' pero no el ')'); esto rompe la validez del resto si y solo si el balance ORIGINAL en algún punto de ese rango ya era 0 (pues bajaría a -1). Como los únicos puntos con balance 0 son los cierres de bloque, el par es INCOMPATIBLE si y solo si i y j pertenecen a bloques distintos. El total de parejas incompatibles se calcula entonces como (total de parejas (i, j) válidas en toda la secuencia) menos (parejas válidas contando solo dentro de cada bloque por separado). Ambas cantidades se obtienen en un único recorrido llevando un contador de '(' abiertos, global y reiniciado por bloque.

Ejercicio aplicado

Dada una secuencia de N bailarines Yang '(' y Yin ')' (garantizada balanceada), donde cada Yang debe emparejarse con un Yin posterior, contar cuántas parejas (i, j) son "incompatibles": aquellas que, de formarse, impiden que el resto de bailarines encuentre pareja.

I/O: $N \leq 10^6 \rightarrow$ un solo recorrido lineal con lectura total (patrón 2); no hace falta tokenizar por espacios ya que la secuencia es un único string contiguo. Verificado: ~ 0.1 s en el peor caso.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    s = data[1].decode()
```

```
total_pairs = 0      # total de parejas (i,j) con i<j,
                      # s[i]='(', s[j]=')'
within_block = 0     # de esas, cuantas quedan DENTRO
                      # del mismo bloque de nivel superior
open_global = 0
open_block = 0
bal = 0
```

```
for ch in s:
    if ch == '(':
        open_global += 1
        open_block += 1
        bal += 1
    else:
        total_pairs += open_global
        within_block += open_block
        bal -= 1
        if bal == 0:      # cierre de un bloque
                           # de nivel superior y reinicia el contador de bloque
            open_block = 0
```

```
print(total_pairs - within_block)
```

solve()

8.2 Rotación Mínima, Booth (huellas duplicadas por rotación)

Preguntas guía:

- ¿Dos strings se consideran "iguales" si uno es una rotación cíclica del otro?
- ¿Necesito una forma canónica única para comparar/deduplicar strings bajo rotación, en tiempo lineal (no $\mathcal{O}(n^2)$ probando todas las rotaciones)?
- ¿El string puede ser largo (miles de caracteres) y hay muchos para procesar?

Utilidad: Encontrar la rotación lexicográficamente menor de un string en $\mathcal{O}(n)$, como forma canónica para comparar o deduplicar strings equivalentes bajo rotación cíclica. Evita el enfoque ingenuo de generar y comparar las n rotaciones ($\mathcal{O}(n^2)$).

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: El algoritmo de Booth adapta la función de fallo (como en KMP) sobre el string duplicado $(s + s)$. Mantiene un índice candidato k a la rotación mínima y, al procesar cada carácter, usa la función de fallo para saltar comparaciones ya conocidas, actualizando k solo cuando encuentra evidencia de que otra posición produce una rotación menor. El resultado equivale a comparar todas las rotaciones en un solo recorrido lineal.

Ejercicio aplicado

Dado un conjunto de huellas dactilares codificadas como strings (equivalentes bajo rotación cíclica), eliminar duplicados representando cada grupo por su rotación canónica (la lexicográficamente menor).

I/O: Hasta 100 strings de hasta 5000 caracteres por caso, múltiples casos hasta el centinela "0" $\rightarrow$ lectura total con `sys.stdin.read().split()` (patrón 2). Verificado: ~ 1.4 s para 5 casos $\times$ 100 strings $\times$ 5000 caracteres, y correcto contra fuerza bruta en 3000 strings aleatorios.

Código solución

```

import sys

def least_rotation(s):
    # indice de inicio de la rotacion lexicograficamente
    # menor (algoritmo de Booth)
    ss = s + s
    n = len(ss)
    fail = [-1] * n
    k = 0
    for j in range(1, n):
        sj = ss[j]
        i = fail[j - k - 1]
        while i != -1 and sj != ss[k + i + 1]:
            if sj < ss[k + i + 1]:
                k = j - i - 1
                i = fail[i]
            if sj != ss[k + i + 1]:
                if sj < ss[k + i + 1]:
                    k = j
                fail[j - k] = -1
            else:
                fail[j - k] = i + 1
    return k

def canonical(s):
    k = least_rotation(s)
    return s[k:] + s[:k]

def solve():
    data = sys.stdin.read().split()
    idx = 0
    out = []
    while idx < len(data):
        n = int(data[idx]); idx += 1
        if n == 0:
            break
        seen = set()
        for _ in range(n):
            m = int(data[idx]); f = data[idx + 1]; idx += 2
            c = canonical(f)
            if c not in seen: # dedup
                # preservando orden de aparicion
                seen.add(c)
                out.append(c)
        sys.stdout.write("\n".join(out) + "\n")
    solve()

```

8.3 Pila con Restricción de Capacidad (matrioshkas)

Preguntas guía:

- ¿La entrada representa una estructura jerárquica donde los elementos se abren y posteriormente deben cerrarse respetando el orden (LIFO)?
- ¿Cada elemento abierto posee una restricción que depende de todos los elementos contenidos directamente en su interior?
- ¿Necesito validar simultáneamente el correcto anidamiento y una condición adicional sobre el contenido de cada nivel?

Utilidad: Validar estructuras anidadas donde cada objeto debe cerrarse exactamente en el orden inverso en que fue abierto y, además, debe cumplir una restricción sobre los elementos contenidos directamente en su interior. Este patrón aparece frecuentemente en problemas de parsing,

validación de expresiones jerárquicas y simulación de estructuras recursivas.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: Cada juguete comienza cuando aparece un entero negativo $-x$ y termina cuando aparece el correspondiente entero positivo x . Debido a que un juguete debe cerrarse antes de continuar cerrando el que lo contiene, el orden de cierre siempre sigue la política **Last In, First Out (LIFO)**, por lo que una pila modela naturalmente la estructura.

Ejercicio aplicado

Dada una secuencia de enteros que representa la apertura y cierre de juguetes generalizados, determinar si la estructura corresponde a una matrioshka válida.

Cada juguete de tamaño m puede contener varios juguetes, pero la suma de los tamaños de todos sus hijos directos debe ser estrictamente menor que m .

I/O: Cada línea representa un caso de prueba independiente. Los valores aparecen separados por espacios y pueden alcanzar magnitudes de hasta 10^7 . Como únicamente se realiza un recorrido secuencial sobre cada línea utilizando una pila, el algoritmo procesa cada caso en tiempo lineal.

Código solución

```

import sys

for line in sys.stdin:
    nums = list(map(int, line.split()))
    stack = []
    valid = True

    for x in nums:
        if x < 0:
            stack.append([-x, 0])
        else:
            if not stack:
                valid = False
                break
            size, total = stack.pop() # numero del
            # cierre y cuantos tiene adentro
            if size != x:
                valid = False
                break
            if total >= size:
                valid = False
                break
            if stack:
                stack[-1][1] += size # del ultimo en
                # en la cola le suma numero

    if stack:
        valid = False
    if valid:
        print(":-) Matrioshka!")
    else:
        print(":( Try again.")

```

8.4 Normalización + Comparación con el Reverso (palíndromos)

Preguntas guía:

- ¿Piden verificar si un texto es un palíndromo ignorando mayúsculas, espacios y puntuación?

- ¿Basta con normalizar el string (filtrar solo los caracteres relevantes, unificar case) y comparar contra su reverso?
- ¿El caso de un string vacío (tras filtrar) debe considerarse un palíndromo válido?

Utilidad: Resolver directamente problemas de verificación de palíndromos con reglas de normalización, filtrando primero y comparando el resultado contra su versión invertida.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: Un string es palíndromo si es igual a su reverso. Al normalizar primero (quedarse solo con letras, todas en minúscula), la comparación directa `filtered == filtered[::-1]` captura exactamente la definición pedida, incluyendo el caso borde de que un string vacío (o sin letras) es trivialmente un palíndromo.

Ejercicio aplicado

Dada una oración, determinar si es un palíndromo ignorando mayúsculas, espacios, y cualquier carácter que no sea una letra del alfabeto inglés.

I/O: Múltiples líneas hasta el centinela "*" (patrón 3), $N < 1000$ por línea $\rightarrow$ se recorre línea por línea con `for line in sys.stdin`, ya que es texto libre (no se puede tokenizar con `.split()`).

Código solución

```
import sys

def solve():
    out = []
    for raw in sys.stdin:
        line = raw.rstrip('\n').rstrip('\r')
        if line == '*':
            break
        filtered = [c.lower() for c in line if c.isalpha()]
        out.append('Y' if filtered == filtered[::-1] else 'N')
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

8.5 Reducción con Pila (ciclos en un bang path)

Preguntas guía:

- ¿Debo procesar una secuencia donde ciertos elementos pueden cancelarse al aparecer nuevamente?
- ¿Eliminar un ciclo puede generar inmediatamente otro ciclo más grande?
- ¿Necesito conservar el orden original mientras simplifico la secuencia al máximo?

Utilidad: Simplificar recorridos o secuencias eliminando ciclos innecesarios. Cada vez que un elemento vuelve a aparecer, todo el recorrido intermedio deja de ser útil y puede eliminarse. Después de cada eliminación es necesario volver a comprobar si ahora se formó un nuevo ciclo con los elementos restantes. Este patrón aparece en problemas de rutas, navegación, directorios, historial de movimientos y simplificación de secuencias.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: La solución utiliza una pila que representa la ruta simplificada construida hasta el momento; cada máquina se procesa exactamente una vez y se inserta al final de la pila. Después de insertar una máquina pueden ocurrir dos situaciones. La primera es si las dos últimas máquinas tienen el mismo nombre donde la última visita es redundante y puede eliminarse. La segunda donde la última máquina coincide con la antepenúltima, por lo que tanto la máquina intermedia como la repetida pueden eliminarse.

Ejercicio aplicado

Dado un bang path de UUCP (UNIX-to-UNIX Copy Program), eliminar todos los saltos innecesarios producidos por ciclos entre máquinas, conservando exactamente el mismo destino final y respetando las mayúsculas y minúsculas de las máquinas que permanecen.

Las comparaciones entre nombres de máquinas son **case-insensitive**, aunque la salida debe conservar la capitalización original.

I/O: La entrada consiste en una única línea cuya longitud no supera los 256 caracteres. Cada componente está separado por el carácter '!' y el último elemento siempre corresponde al usuario destino, por lo que nunca participa en las simplificaciones. Debido al pequeño tamaño de la entrada, basta dividir el string utilizando `split("!!")` y recorrer los componentes una única vez.

Código solución

```
import sys
input = sys.stdin.buffer.readline
data = input().decode("utf-8").split("!!") # Separa cada
componente del bang path

if len(data) <= 2: # Si no existen maquinas intermedias
    no hay nada que simplificar
    sys.stdout.write("!!".join(data))
    exit()

machines = data[:-1] # todas las maquinas
user = data[-1] # usuario final (no se modifica)
stack = []

for machine in machines:
    stack.append(machine) # agrega la maquina actual
    a la ruta
    while True: # mientras aparezcan nuevos ciclos
        despues de eliminar otros
        if (len(stack) >= 2 and stack[-1].lower() ==
            stack[-2].lower()): #CASO AA
            stack.pop()
            continue

        if (len(stack) >= 3 and stack[-1].lower() ==
            stack[-3].lower()): #CASO ABA
            stack.pop() # elimina la ultima A
            stack.pop() # elimina B
            continue
        break # ya no existen reducciones posibles
sys.stdout.write("!!".join(stack + [user])) # reconstruye
el bang path conservando el usuario final
```

8.6 KMP sobre Codificación en Preorden (subárboles con la misma forma)

Preguntas guía:

- ¿Piden contar/encontrar nodos de un árbol cuyo subárbol tiene la MISMA FORMA que otro árbol patrón, ignorando por completo las etiquetas (valores, nombres de variable, operadores)?
- ¿Los árboles son binarios completos (todo nodo interno tiene exactamente 2 hijos) y el orden izquierda/derecha de los hijos SÍ importa para la comparación?
- ¿Los árboles vienen dados como texto con una gramática recursiva fácil de recorrer linealmente (ej. expresiones totalmente parentizadas)?
- ¿|patrón| y |texto| son grandes (decenas o cientos de miles), descartando comparar subárbol por subárbol de forma ingenua?

Utilidad: Reduce un problema de comparación estructural de árboles (¿este subárbol tiene la misma forma que aquel otro?) a un problema clásico de coincidencia de patrones en cadenas. Es aplicable a cualquier familia de árboles binarios completos donde solo interesa la forma (posición y arity de los nodos), no las etiquetas: sirve tanto para árboles de expresión como para cualquier otra estructura jerárquica que se pueda serializar de la misma manera.

Complejidad: $\mathcal{O}(|\alpha| + |\beta|)$ por caso de prueba, tanto para la codificación como para la búsqueda de coincidencias con KMP.

Fundamento teórico: Todo árbol binario completo (cada nodo interno tiene exactamente 2 hijos, las hojas 0) admite una codificación en preorden como una cadena binaria: cada hoja emite un símbolo '0' y cada nodo interno emite un símbolo '1' seguido de la codificación de su hijo izquierdo y luego la de su derecho. Esta codificación es una biyección: dos árboles son estructuralmente idénticos si y solo si sus cadenas codificadas son idénticas carácter a carácter. Además, como el recorrido es en preorden, la codificación del subárbol enraizado en cualquier nodo v es exactamente una SUBCADENA CONTIGUA de la codificación completa, que comienza en la posición de v y cuya longitud es el tamaño de ese subárbol — esto se cumple porque el proceso de decodificación es determinista (depende únicamente de la secuencia de bits leída, nunca de lo que venga después), así que si dos subcadenas de igual longitud L son iguales carácter a carácter, sus procesos de decodificación son idénticos paso a paso y por tanto ambas "cierran" el árbol exactamente tras consumir L símbolos, produciendo árboles idénticos. Como además cada nodo del árbol contribuye exactamente un símbolo en el recorrido preorden, TODAS las posiciones de la cadena de texto corresponden a un nodo válido, así que contar nodos v cuyo subárbol coincide con el patrón α se reduce exactamente a contar cuántas veces aparece (con solapamiento permitido) la cadena codificada de α dentro

de la cadena codificada de β , lo cual se resuelve en tiempo lineal con el autómata de fallos de Knuth-Morris-Pratt (KMP), reanudando la búsqueda en `fail[m-1]` tras cada coincidencia para permitir solapamientos.

Ejercicio aplicado

Dadas expresiones aritméticas α y β totalmente parentizadas (operadores binarios, variables de una letra, números), contar cuántos nodos del árbol de sintaxis de β tienen un subárbol con la misma forma que el árbol de α , ignorando etiquetas (variables, números y operadores).

I/O: Varios casos de prueba hasta EOF, cada uno de dos líneas sin espacios internos, con $1 \leq |\alpha|, |\beta| \leq 400000 \rightarrow$ como ninguna expresión contiene blancos, se usa lectura total + tokenización (patrón 2: `sys.stdin.buffer.read().split()`), donde cada token es directamente una expresión completa. La codificación se hace con un único barrido lineal por carácter (sin recursión ni pila, evitando cualquier límite de profundidad de Python) y la búsqueda usa KMP clásico. Verificado contra los ejemplos dados (salidas 3 y 2) y contrastado con un verificador de fuerza bruta (parseo real del árbol + comparación de formas) en 300 casos aleatorios pequeños, sin discrepancias. Con un caso cercano al peor caso teórico ($|\alpha| = |\beta| \approx 400000$) el tiempo medido fue de 0.11s, y con varios casos de prueba encadenados (texto total >1.6 MB) 0.2s, muy por debajo de límites típicos.

Código solución

```
import sys

def encode(tok):
    # codifica la expresion (bytes) en preorden: '1' =
    # nodo interno, '0' = hoja
    # un solo barrido lineal, sin recursion ni pila
    out = bytearray()
    i, n = 0, len(tok)
    ap = out.append
    while i < n:
        c = tok[i]
        if c == 40:                # '(' -> abre un nodo
            interno
            ap(49)                 # emite '1'
            i += 1
        elif c == 41:             # ')' -> solo delimita,
            no aporta simbolo
            i += 1
        elif c == 43 or c == 45 or c == 42 or c == 47: #
            + - * / (operador)
            i += 1
        elif 97 <= c <= 122:      # variable (una sola
            letra a-z)
            ap(48)                 # emite '0' (hoja)
            i += 1
        else:                     # numero: puede tener
            varios digitos, es UNA sola hoja
            ap(48)
            i += 1
            while i < n and 48 <= tok[i] <= 57:
                i += 1
    return bytes(out)

def kmp_count(pattern, text):
    # cuenta ocurrencias de pattern en text, permitiendo
    # solapamiento
    m = len(pattern)
```

```

if m == 0 or m > len(text):
    return 0
fail = [0] * m
k = 0
for i in range(1, m):
    while k > 0 and pattern[i] != pattern[k]:
        k = fail[k - 1]
    if pattern[i] == pattern[k]:
        k += 1
    fail[i] = k
count = 0
k = 0
for c in text:
    while k > 0 and c != pattern[k]:
        k = fail[k - 1]
    if c == pattern[k]:
        k += 1
    if k == m:
        count += 1
        k = fail[k - 1]    # permite coincidencias
    solapadas
return count

def solve():
    data = sys.stdin.buffer.read().split()
    out = []
    for i in range(0, len(data), 2):
        alpha, beta = data[i], data[i + 1]
        out.append(str(kmp_count(encode(alpha), encode(beta))))
    sys.stdout.write("\n".join(out) + "\n")

solve()

```

8.7 Suffix Array + LCP de Kasai (subcadena repetida más larga)

Preguntas guía:

- ¿Me piden la subcadena más larga que aparece al menos dos veces en un texto (permitiendo solapamiento)?
- ¿Necesito comparar prefijos comunes entre TODAS las subcadenas del texto, y enumerarlas explícitamente sería cuadrático o peor?
- ¿El problema se reduce a una consulta sobre prefijos comunes de sufijos (repeticiones, subcadena común, cantidad de subcadenas distintas)?

Utilidad: El suffix array ordena lexicográficamente todos los sufijos de una cadena, y el arreglo LCP guarda la longitud del prefijo común entre cada par de sufijos ADYACENTES en ese orden. Juntos permiten responder en tiempo casi lineal preguntas sobre repeticiones y subcadenas comunes que de forma ingenua requerirían enumerar las $\mathcal{O}(n^2)$ subcadenas: subcadena repetida más larga, cantidad de subcadenas distintas, subcadena común más larga entre varios textos, y consultas de prefijo común entre sufijos arbitrarios.

Complejidad: $\mathcal{O}(n \log^2 n)$ para construir el suffix array por duplicación (log n rondas, cada una con un ordenamiento), y $\mathcal{O}(n)$ amortizado para el arreglo LCP con el algoritmo de Kasai.

Fundamento teórico: Toda subcadena de S es un PREFIJO de algún sufijo de S , así que una subcadena que aparece dos veces corresponde a un prefijo

común de dos sufijos distintos, y su longitud máxima es $\max_{i \neq j} \text{lcp}(\text{suf}_i, \text{suf}_j)$. La clave es que no hace falta examinar los $\mathcal{O}(n^2)$ pares: al ordenar los sufijos lexicográficamente, la función lcp cumple $\text{lcp}(\text{sa}_i, \text{sa}_j) = \min_{i < t \leq j} \text{lcp}(\text{sa}_{t-1}, \text{sa}_t)$ para $i < j$ (un mínimo sobre un rango del arreglo LCP), porque un prefijo común a dos sufijos lo es también de todos los sufijos intercalados entre ellos en el orden. Como un mínimo sobre un rango nunca supera a ninguno de sus elementos, el máximo global sobre todos los pares se alcanza necesariamente en un par ADYACENTE, y la respuesta es simplemente $\max(\text{lcp})$. La construcción del suffix array por duplicación se apoya en que, conociendo el rango de cada sufijo por sus primeros k caracteres, el rango por $2k$ caracteres se obtiene ordenando por el par $(\text{rank}_k[i], \text{rank}_k[i + k])$ — de ahí las $\lceil \log n \rceil$ rondas. El algoritmo de Kasai calcula los LCP en orden de POSICIÓN en el texto (no de rango) usando el invariante de que, si lcp del sufijo que empieza en i vale h , entonces el del sufijo que empieza en $i + 1$ vale al menos $h - 1$: al quitar el primer carácter de dos cadenas que comparten h caracteres, siguen compartiendo $h - 1$. Por eso h solo decrece en una unidad por iteración pero puede crecer arbitrariamente, dando un total amortizado de $\mathcal{O}(n)$.

Ejercicio aplicado

Dados T textos, para cada uno hallar la longitud de la subcadena más larga que aparece al menos dos veces en él (las dos apariciones pueden solaparse).

I/O: T casos, cada uno una cadena en su propia línea con $|S| \leq 5000$ sobre el alfabeto minúsculo $\Rightarrow$ como las cadenas no contienen espacios, se usa lectura total + tokenización (patrón 2) y cada token es directamente una cadena completa. Se atiende aparte el caso $|S| < 2$ (respuesta 0: no puede haber repetición). Verificado contra el caso de ejemplo y contrastado con una fuerza bruta (enumerar todas las subcadenas de cada longitud en un conjunto) en 158 casos, incluyendo alfabeto binario, cadenas de un solo carácter repetido y otros casos borde, sin discrepancias. El enunciado no acota T , así que se midió el peor caso razonable: una cadena de $|S| = 5000$ tarda $\sim 0.02\text{s}$ (tanto aleatoria como en el peor caso para Kasai, "a" repetida 5000 veces), y $T = 100$ cadenas de longitud máxima $\sim 1.04\text{s}$.

Código solución

```

import sys

def suffix_array(s):
    n = len(s)
    # ronda inicial: ordenar los sufijos por su primer
    # caracter
    sa = sorted(range(n), key=lambda i: s[i])
    rank = [0]*n
    r = 0
    prev = s[sa[0]]
    for idx, i in enumerate(sa):
        if s[i] != prev:
            r += 1
            prev = s[i]
        rank[i] = r

```

```

k = 1
# duplicacion: si ya se conoce el rango por k
# caracteres,
# ordenar por el par (rank[i], rank[i+k]) da el rango
# por 2k
while r < n - 1:
    # corta apenas todos
    # los rangos son distintos
    key = [0]*n
    for i in range(n):
        # el par se empaqueta en un solo entero para
        # ordenar mas rapido
        key[i] = rank[i] * (n + 1) + (rank[i+k] + 1)
    if i + k < n else 0)
    sa.sort(key=lambda i: key[i])
    new_rank = [0]*n
    r = 0
    prev = key[sa[0]]
    for i in sa:
        if key[i] != prev:
            r += 1
            prev = key[i]
        new_rank[i] = r
    rank = new_rank
    k <= 1
return sa, rank

def kasai(s, sa, rank):
    n = len(s)
    lcp = [0]*n
    h = 0
    # se recorre por POSICION en el texto: al avanzar de
    # i a i+1,
    # el lcp baja como mucho en 1, asi que h nunca se
    # reinicia a 0
    for i in range(n):
        if rank[i] > 0:
            j = sa[rank[i]-1]
            # sufijo anterior en
            # orden lexicografico
            while i + h < n and j + h < n and s[i+h] == s
            [j+h]:
                h += 1
            lcp[rank[i]] = h
            if h:
                h -= 1
        else:
            h = 0
    return lcp

def solve():
    data = sys.stdin.buffer.read().split()
    t = int(data[0])
    out = []
    for i in range(1, t+1):
        s = data[i].decode()
        n = len(s)
        if n < 2:
            out.append("0")
            continue
        sa, rank = suffix_array(s)
        lcp = kasai(s, sa, rank)
        # el maximo lcp sobre pares adyacentes ya es el
        # maximo global
        out.append(str(max(lcp)))
    sys.stdout.write("\n".join(out) + "\n")

solve()

```

Extras: variantes de `solve()` para cuando el problema pide, además de la longitud, QUÉ subcadena es y DÓNDE aparece (`lcp_detailed`: todas las subcadenas de longitud máxima con sus posiciones) o cuál es la subcadena más larga que aparece el MAYOR número de

veces (`lcp_repeated`, con mínimo en ventana deslizante sobre el arreglo LCP). Se llaman con `(s, sa, lcp, n)` en lugar de `max(lcp)`.

```

from collections import deque, Counter

def blocks(s, sa, lcp, n, L):
    # bloques maximales con lcp >= L -> (subcadena,
    # ocurrencias)
    res = []
    p = 1
    while p < n:
        if lcp[p] >= L:
            ini = p
            while p < n and lcp[p] >= L:
                p += 1
            occ = sorted(sa[ini-1:p])
            res.append((s[sa[ini]:sa[ini]+L], occ))
        else:
            p += 1
    return res

def lcp_detailed(s, sa, lcp, n):
    L = max(lcp)
    if L == 0:
        return "0"
    partes = [f'"{sub}" x{len(occ)} en {occ}' for sub,
    occ in blocks(s, sa, lcp, n, L)]
    return f"{L} -> " + " | ".join(partes)

def lcp_repeated(s, sa, lcp, n):
    # ninguna subcadena se repite mas que su primer
    # caracter
    c = max(Counter(s).values())
    if c < 2:
        return "0"
    w = c - 1
    # ventana de w valores
    # de lcp = c sufijos
    dq = deque()
    # minimo de ventana
    # deslizante
    mejorL = mejorA = -1
    for p in range(1, n):
        while dq and lcp[dq[-1]] >= lcp[p]:
            dq.pop()
        dq.append(p)
        if dq[0] < p - w + 1:
            dq.popleft()
        if p >= w and lcp[dq[0]] > mejorL:
            mejorL = lcp[dq[0]]
            mejorA = p - w + 1
    occ = sorted(sa[mejorA-1:mejorA+w])
    return f"{mejorL} -> '{s[sa[mejorA]:sa[mejorA]+mejorL
    ]}' x{len(occ)} en {occ}"

```

8.8 Firma Canónica + Hash Map (palabras con letras revueltas)

Preguntas guía:

- ¿Debo agrupar o emparejar palabras que son "la misma" bajo alguna transformación permitida (anagramas, reordenamientos parciales, rotaciones, cambios de caso)?
- ¿Puedo definir una función que colapse todas las palabras equivalentes al MISMO valor, sin comparar cada par entre sí?
- ¿Me piden, entre todos los candidatos equivalentes, el menor lexicográficamente (o algún otro criterio de desempate)?

- ¿Comparar todas las palabras del texto contra todas las del diccionario sería cuadrático o más?

Utilidad: Convierte un problema de "¿son equivalentes estas dos palabras?" en un problema de búsqueda exacta en tabla hash. En lugar de comparar cada palabra del texto contra cada palabra del diccionario, se define una FIRMA CANÓNICA: una función que asigna a todas las palabras de una misma clase de equivalencia un valor idéntico, y a palabras de clases distintas valores distintos. El diccionario se indexa una sola vez por firma, y cada consulta se resuelve con un acceso directo. Aplica a anagramas, reordenamientos parciales, normalización de cadenas y, en general, a cualquier relación de equivalencia que se pueda describir por un conjunto de invariantes.

Complejidad: $O(D \log L + T)$ amortizado por caso de prueba, donde D es el número de palabras del diccionario, T el del texto y L la longitud máxima de palabra (el $\log L$ viene de ordenar las letras del medio para construir la firma). Sin la firma, la comparación directa sería $O(D \cdot T \cdot L \log L)$.

Fundamento teórico: La transformación permitida — reordenar todas las letras excepto la primera y la última — genera una relación de equivalencia sobre las palabras. Dos palabras son equivalentes si y solo si coinciden en CUATRO invariantes: la longitud, la primera letra, la última letra y el MULTICONJUNTO de letras intermedias. Estos invariantes son completos: si coinciden, existe una permutación de las letras del medio que transforma una en la otra (basta reordenar el multiconjunto), y si difieren en alguno, ninguna permutación del medio puede igualarlas (porque una permutación conserva el multiconjunto y no toca los extremos ni la longitud). Por lo tanto, la tupla $(|w|, w_0, w_{-1}, \text{sort}(w_{1..-2}))$ es una forma canónica: dos palabras tienen la misma firma exactamente cuando son intercambiables. La longitud debe incluirse explícitamente: sin ella, las palabras "a" y "aa" producirían la misma firma ('a', 'a', "") — en la primera la única letra hace de primera Y de última, y en ambas el medio es vacío — a pesar de tener longitudes distintas y no ser intercambiables. Para el desempate, como la respuesta pedida es el menor lexicográficamente, basta conservar en cada casilla de la tabla el mínimo visto; nótese que la palabra correcta puede NO ser la propia palabra del texto aunque esta esté en el diccionario, como muestra el segundo caso de ejemplo, donde `drinaemg` pertenece al diccionario pero se reemplaza por `dreaming` por ser anterior en orden lexicográfico.

Ejercicio aplicado

Dado un diccionario y un texto cuyas palabras tienen las letras del medio revueltas (primera y última en su lugar original), reemplazar cada palabra del texto por la palabra del diccionario que se convierte en ella reordenando solo las letras intermedias, eligiendo la menor lexicográficamente si hay varias, o dejándola igual si no hay ninguna.

I/O: N casos, cada uno en EXACTAMENTE dos líneas (diccionario y texto) de a lo sumo 200 caracteres cada una

⇒ la estructura por líneas es semánticamente relevante (hay que saber dónde termina el diccionario y empieza el texto), así que se usa `sys.stdin.buffer.readline` línea por línea (patrón 1) y se tokeniza cada línea con `.split()`. Se trabaja directamente sobre `bytes` sin decodificar, ya que la firma solo necesita comparar e indexar; solo la salida se decodifica. Los límites son diminutos (200 caracteres por lista), así que el rendimiento no es un problema, aunque N no está acotado superiormente en el enunciado. Verificado contra los dos casos de ejemplo y contrastado con una fuerza bruta (comparar cada palabra del texto contra todo el diccionario) en 600 casos aleatorios, sin discrepancias, incluyendo casos borde de palabras de 1, 2 y 3 letras.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def firma(w):
    # dos palabras son intercambiables si coinciden
    # longitud, primera letra,
    # ultima letra y el MULTICONJUNTO de letras del medio
    # la longitud es imprescindible: sin ella "a" y "aa"
    # colisionarian
    return (len(w), w[0], w[-1], bytes(sorted(w[1:-1])))

def solve():
    n = int(input())
    out = []
    for _ in range(n):
        dic = input().split()
        txt = input().split()
        mejor = {}
        for d in dic:
            f = firma(d)
            # conserva la menor lexicograficamente (el
            # diccionario ya viene ordenado,
            # pero la comparacion explicita lo hace
            # independiente del orden de entrada)
            if f not in mejor or d < mejor[f]:
                mejor[f] = d
            # si ninguna palabra del diccionario aplica, se
            # deja la original
        res = [mejor.get(firma(w), w) for w in txt]
        out.append(b" ".join(res).decode())
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

8.9 Regex: findall y lookahead (conteo de patrones)

Preguntas guía:

- ¿El problema se reduce a contar cuántas veces aparecen unos pocos patrones LITERALES dentro de un texto, sin estructura algorítmica adicional?
- ¿Los patrones son fijos y conocidos de antemano, de modo que puedo hacer una pasada independiente por cada uno?
- ¿Debo verificar si los patrones pueden SOLAPARSE consigo mismos, para decidir entre conteo simple o lookahead?
- ¿El texto es suficientemente corto como para que el costo de varias pasadas sea irrelevante?

Utilidad: Resolver directamente, con el módulo `re` de la librería estándar, problemas cuya esencia es localizar o contar ocurrencias de patrones en un texto. Para patrones literales y fijos evita escribir a mano un autómata o un KMP: el motor de expresiones regulares ya está optimizado en C y una sola línea reemplaza decenas. Además, `re` da acceso gratuito a posiciones de coincidencia, sustitución, partición del texto y conteo con solapamiento, que suelen ser justamente las variantes que pide un problema parecido.

Complejidad: $\mathcal{O}(kn)$ donde n es la longitud del texto y k la cantidad de patrones, ya que se hace una pasada lineal por patrón (el motor recorre el texto una vez por cada llamada a `findall`).

Fundamento teórico: Contar ocurrencias de un patrón literal admite dos semánticas distintas y el motor de `re` implementa solo una: `re.findall` avanza el cursor hasta el FINAL de cada coincidencia, de modo que nunca devuelve coincidencias solapadas. La pregunta relevante es entonces cuándo esa restricción cambia el resultado, y la respuesta es puramente estructural: un patrón P puede solaparse consigo mismo si y solo si tiene un BORDE PROPIO, es decir, un prefijo propio que también es sufijo. Si P tiene un borde de longitud $b > 0$, dos ocurrencias pueden empezar a distancia $|P| - b$ y compartir b caracteres; si no tiene ningún borde, dos ocurrencias distintas no pueden compartir ni un carácter, y el conteo sin solapamiento coincide exactamente con el conteo total. Los tres patrones de este problema (`bravo`, `ha`, `boooo`) no tienen borde propio — ninguno de sus prefijos propios coincide con un sufijo — así que `findall` es correcto sin más cuidados. En contraste, `aa` tiene borde `a` y en `aaaa` `findall` devuelve 2 mientras que las ocurrencias reales son 3. Cuando sí hace falta contar solapadas, se usa un LOOKAHEAD (`?=P`): al ser una aserción de ancho cero, no consume caracteres, así que el motor avanza una sola posición tras cada éxito y encuentra todas las ocurrencias; devuelve cadenas vacías, pero lo que interesa es cuántas son. El otro punto no evidente es que hacer una pasada separada por patrón NO equivale a una única pasada con alternancia `P1|P2|P3`: la alternancia consume el texto, así que los patrones compiten entre sí y una coincidencia larga puede ocultar a otra contenida en ella (en `bravo`, el patrón `bravo|ra` encuentra solo `bravo`, perdiendo el `ra` interior). Como el enunciado cuenta cada patrón de forma independiente, las pasadas separadas son la traducción fiel.

Ejercicio aplicado

Dada una cadena de sonidos del público, calcular una puntuación sumando 3 por cada aparición de `bravo`, sumando 1 por cada `ha` y restando 1 por cada `boooo`.

I/O: Una única línea con la cadena, $|S| \leq 1000 \Rightarrow$ lectura de una sola línea con `sys.stdin.buffer.readline` (patrón 1), con `.strip()` para quitar el salto de línea y `.decode()` porque `re` trabaja sobre `str`. El tamaño es diminuto, así que las tres pasadas independientes no representan ningún costo. Verificado contra los cinco casos

de ejemplo del enunciado, y comprobado aparte que ninguno de los tres patrones tiene borde propio, contrastando además `findall` contra el conteo con lookahead en 20 000 cadenas aleatorias sobre el alfabeto relevante sin una sola diferencia: por eso la ausencia de manejo de solapamiento es correcta y no una omisión.

Código solución

```
import re
import sys
input = sys.stdin.buffer.readline

data = input().strip().decode() # re trabaja sobre str,
                                # no sobre bytes

pat1 = r'bravo'
pat2 = r'ha'
pat3 = r'boooo'

score = 0
# findall NO cuenta solapamientos, pero ninguno de estos
# tres patrones
# tiene borde propio (prefijo == sufijo), así que no
# puede solaparse consigo mismo
score += 3*len(re.findall(pat1, data))
score += len(re.findall(pat2, data))
score -= len(re.findall(pat3, data))

sys.stdout.write(str(score)+"\n")
```

Extras: referencia rápida de `re` — no forman parte de la solución, son variantes útiles para problemas parecidos. Todos los resultados mostrados fueron ejecutados con `data = "boooohaboooo"`.

```
import re
data = "boooohaboooo"

# POSICIONES E INFORMACION DE CADA OCURRENCIA
for m in re.finditer(r'ha', data):
    m.group() # 'ha' el texto que coincidio
    m.start() # 5 índice inicial
    m.end() # 7 índice final (exclusivo)
    m.span() # (5, 7) la tupla (inicio, fin)

# SOLAPAMIENTO
# r'oo' CONSUME caracteres -> no solapa
re.findall(r'oo', data) # ['oo', 'oo', 'oo', 'oo']
# -> 4
# r'(?=oo)' es un lookahead: ancho cero, NO consume -> si
# solapa
re.findall(r'(?=oo)', data) # ['', '', '', '', '', '']
# -> 6
[m.start() for m in re.finditer(r'(?=oo)', data)] #
[1, 2, 3, 8, 9, 10]

# GRUPOS DE CAPTURA
m = re.search(r'(b)(o+)', data) # search: primera
# coincidencia o None
m.group(0) # 'boooo' la coincidencia completa
m.group(1) # 'b' primer parentesis
m.group(2) # 'oooo' segundo parentesis
m.groups() # ('b', 'oooo')
# CUIDADO: si el patron tiene grupos, findall devuelve
# TUPLAS, no la coincidencia
re.findall(r'(b)(o+)', data) # [('b', 'oooo'), ('b', '
# oooo')]
re.findall(r'(?:(b)?(o+))', data) # ['boooo', 'boooo']
# (?: ) agrupa sin capturar

# SUSTITUCION Y PARTICION
```

```

re.sub(r'o+', '0', data)      # 'b0hab0'
re.subn(r'o+', '', data)     # ('bhab', 8) devuelve (
    texto, cuantas)
re.split(r'ha', data)        # ['boooo', 'boooo']
re.split(r'(ha)', data)      # ['boooo', 'ha', 'boooo']
    con grupo conserva el separador

# COMPILAR (si el patron se reutiliza muchas veces)
rx = re.compile(r'boooo')
rx.findall(data)              # ['boooo', 'boooo']

# UNA SOLA PASADA CON ALTERNANCIA
puntos = {'bravo':3, 'ha':1, 'boooo':-1}
rx2 = re.compile(r'bravo|boooo|ha')
sum(puntos[x] for x in rx2.findall(data)) # -1,
    coincide aqui
# PERO la alternancia CONSUME, asi que los patrones
    compiten entre si:
re.findall(r'bravo|ra', 'bravo') # ['bravo'] <- se
    pierde el 'ra' interior
re.findall(r'ra', 'bravo')       # ['ra']
# por eso, si el enunciado cuenta cada patron por
    separado, van pasadas separadas

# PATRON LITERAL CON CARACTERES ESPECIALES
re.escape('a.b*c')              # 'a\\.b\\*c' neutraliza .
    * + ? [ ] ( ) etc.

```